

Go与Java

零.GoApi文档和中文社区网址

Go的中文api文档: <https://studygolang.com/pkgdoc>

Go中文社区网站: <https://studygolang.com/>

一.关于Java

1.Java的用途

首先我们来回顾下Java的主要用途和应用场景:

用途一: 服务器后端系统开发(web后端、微服务后端支付系统、业务系统、管理后台, 各种后台交互的接口服务)。

用途二: 大数据框架的底层实现和Java的API支持。(Hadoop)。

用途三: 其它中间件的底层开发。(Tomcat、RocketMq、Hbase、Kafka(部分)、SpringCloud, Dubbo...)。

2.Java的优势和特点

那我们看Java语言有什么优势和特点呢?

- *. 做服务端系统性能高。
- *. 有虚拟机, 跨平台。
- *. 功能强大, 支持的类库多, 生态圈类库多, 开发框架和工具更易找。
- *. 市场占有率高, 约60%的中国程序员都是做Java相关的工作。

二.关于Go

1.Go的出生原因

Go语言是Google内部公司大佬开发的, 主要起因于Google公司有大量的C程序项目, 但是开发起来效率太低, 维护成本高, 于是就开发了Go语言来提高效率, 而且性能只是差一点。

(Go是2007年开始研发, 2009推出发布)

2.宏观看Go与Java的差异

接着，我们来看一下Go语言与Java的差异之处：

- *.无虚拟机，不跨平台(这里的平台指操作系统)(可以运行多个平台，每个平台打不同的二进制程序包)，需要打包编译成对应服务器操作系统版本(windows/linux)的可执行程序(比如windows是exe)。(注：说go跨平台的是指32位和64位相同操作系统之间的跨平台)
- *.因为Go程序直接打包成操作系统可执行的文件，没有虚拟机在中间转换的一层，所以理论上执行效率会更高（理论上更高，实际情况需具体分析）。
- *.相比Java的语言和代码编写风格，Go更简洁，可以用更少的代码实现同样的功能。
- *.Go语言底层也是C实现的，又做了高并发的设计(Java出生时(1995)还没有多核cpu,所以他的并发支持后来添加上去的,Go(2009)出生时已经有了多核cpu的电脑，它在设计语言时就考虑了充分利用多核cpu(英特尔2005首次推出多核)的性能)，所以性能高，高并发的支持(高并发支持其中指的一个就是充分利用多核cpu的性能资源，比如go程序默认使用所有cpu(除非自己设置使用多少))也好。
- *.天然的适用一些特定系统的开发，比如区块链类系统(如以太坊底层系统、以太坊上层应用程序)，云计算和容器(Docker, K8s底层都是go开发的)开发的(大公司自研运维管理项目也大多是用go做底层的开发)，网络编程(类似于java的Netty)。

3.Go和Java的语言类型区别

计算机编程语言按照运行的方式可以分为编译型编程语言和解释型编程语言。

我来举一个例子，你要教别人一门沟通交流的语言，比如英语。

编译型的教的方式就是录(这里的录相当于计算机中把程序编译成二进制可执行文件)一个视频课程，语音课程，把每一句英语发音录下来，这样学生学的时候只要播放你的录音，然后跟着读就行，你只需要录制一次，学生就可以无数次听。

解释性的教的方式就是你亲自到学生家里给他补习，你当面教他，你读(读相当于每次执行都重新用解释器解释一遍)一句他学一句，

这样的话，你想要教他一句你必须就得先读一句，每次教都得重新一遍一遍的读。

这两种教学方式还有一个差别，你录(编译)视频语音教他，你录的英语他就只能学英语，空间环境一变，他现在要去日本，要学日语，你的视频语音教程因为已经录好了，是英语类型(英语类型类比操作系统类型)的，所以，你就得再录一套日语的语音教程。

而现场教他，你也会日语的话，你只需要读(读相当于解释器解释)日语给他听，让他学就行了，是不用考虑语言环境(操作系统类型环境)不同的问题的。

现在我们再来看编程语言，我们的程序执行有两种方式，一种是编译成操作系统可执行的二进制可执行程序，这样相当于编译一次，之后每次执行都不用再编译了，但是因为不同操作系统对于二进制文件的执行规范不同，不同的操作系统你要编译成不同的可执行文件。

解释型语言就是多了一个解释器，解释器我们可以类比为老师，你执行一行代码我们类比为学一句话的读音，解释器解释一句，就是老师先读一句，你跟着才能读一句，也就是解释器每解释一行代码为可运行的代码，操作系统执行一行代码，这样的话每次执行都需要解释器重新解释一遍，执行几次就得解释几次。

Go是编译型的语言，运行在不同的平台需要打包成不同操作系统类型下的可执行文件。

Java是半编译，半解释型语言。编译是指他的代码都会编译成class类型的文件，class类型的文件只需要编译一次，可以在不同的操作系统的Java虚拟机上执行，半解释是指在Java虚拟机中，他还是需要一句一句的将class的二进制代码解释成对应操作系统可执行的代码。

4.Go语言目前的主要应用场景

*.和Java一样，Go语言最多的应用场景就是服务器后端系统的开发，包括web后端，微服务后端接口。

*.Go非常适用需要高性能高并发的网络编程，这里的网络编程是指不需要界面，底层只是用Socket相互传输数据的系统，类似于Java中Netty的用途。

*.一些云计算容器，比如Docker，K8s，底层就是Go语言开发的，也可以用做底层自研运维项目的开发。

*.一些游戏系统的开发，可以用Go语言。

*.区块链的一些底层软件和一些应用软件。（区块链程序的第一开发语言）

5.现在市场上都有哪些公司在使用Go语言？

我们不讲虚的，直接BOSS直聘看哪些公司招，招的是干什么系统开发的。

这是腾讯的一个岗位。

go开发工程师 [深圳·南山区]

20-40K·16薪 3-5年 | 本科

张女士 | 招聘经理

腾讯

互联网 | 已上市 | 10000人以上



Go语言

多线程编程

大开发

linux开发环境

补充医疗保险，节日福利，员工旅游，带薪年假，交通补...

看看岗位描述，是做互联网保险 产品的业务系统开发，业务系统是啥意思，和JAVA后端业务系统一样啊，说明腾讯的一部分项目已经用Go来开发业务系统了，至少他这个保险团队是这样的。

职位描述

岗位职责：

- 1、负责互联网保险产品的业务系统开发和维护；
- 2、负责相关系统的架构设计和开发；
- 3、研究中间件，进行合适的选型引入和对接。

任职要求：

- 1、计算机或相近专业, 大学本科及以上学历, 2年及以上计算机相关工作经验；
- 2、精通Go语言、C++编程, 熟悉Linux开发环境, 对网络编程、多线程编程等有丰富的经验；
- 3、熟悉mysql、redis等中间件, 熟悉数据结构设计和SQL调优；
- 4、有灵敏的技术嗅觉, 能够独立解决问题, 具备清晰的思维和良好的沟通能力；
- 5、有钻研精神, 对技术充满热情；有团队合作精神；充分认可互联网保险的行业价值；
- 6、有高性能、大并发后台服务架构设计和开发经验者优先。

公司介绍

*（有兴趣的同学可以直接发送简历提高效率，看到即回复）

再看小米也是：

Go开发工程师 [北京·海淀区]

18-35K 不限 | 本科 立即沟通

小米

互联网 | 已上市 | 10000人以上

Go语言

消息队列

并发编程

开源项目

Kafka

定期体检，年终奖，补充医疗保险，交通补助，餐补，节...

也是后端，这是要和JAVA抢饭碗。。。

职位描述

职位描述

服务器后端架构的规划和开发;

服务器后端接口设计和开发。

职位要求

计算机相关专业，大学本科学历以上;

具备良好的计算机基础知识，熟悉常用的数据结构和算法;

3年以上Go/Java/Python服务端程序开发经验，熟悉并发编程相关基础知识;

熟练掌握Go语言相关技术，对进程/线程/协程有清晰的概念，熟悉channel和锁;

熟练使用Linux常用命令和Git代码管理工具;

熟练使用MySQL/MongoDB/Redis;

熟悉Kafka等消息队列;

熟练阅读英文文档;

有较强的逻辑思维能力，良好的沟通协调能力和执行推动能力。

优先条件

有高并发或高可用项目经验者优先。

再看一个常见的，Go非常适合开发运维管理系统，这个估计是开发维护阿里内部的自动化运维项目的，也就是说他们的运维支持可能是他们自己用Go语言写的项目。(实在不理解你就想下他们自己自研开发了一个类似于Jenkins和Docker之类的环境和代码流程发布的项目)

职位描述

职位描述

1. 负责阿里云面向企业客户的运维工具体系建设，帮助客户提升自动化运维效率；
2. 搭建完备的技术风险防控工具体系，提升企业客户业务的高可用能力；
3. 面向企业客户需求，打造完善的运营服务体系；
4. 制定标准化服务方案，协同生态合作伙伴，支撑企业服务输出实施落地。

职位要求

1. 计算机软件或相关专业，本科或以上学历；
2. 三年以上 Java/Go 相关开发经验，扎实的编程基础，具备良好的编码风格和单元测试习惯；
3. 具备较好的的抽象设计能力，熟练应用常用的设计模式；
4. 精通分布式系统原理，具备大型的分布式系统的研发、设计、架构经验；
5. 有大规模运维系统设计和高可用体系建设经验；
6. 有前端 React 开发经验者优先，对云有深刻理解，有帮助企业数字化转型经验优先。

公司介绍

阿里巴巴集团的使命是让天下没有难做的生意。

再来看一个字节跳动的，也是开发内部流程自动部署自动运维程序的

职位描述

职位描述

- 1、主要负责项目流程自动化/工程效率等平台的开发工作，包含平台整体架构设计与研发，串联前端、底层需求，负责平台的完整研发流程；
- 2、对架构、前沿技术及研发流程充满兴趣，能够深入调研公司研发体系和流程，给出相应的底层服务的平台支撑方案；
- 3、关注前沿技术研究，通过新技术服务团队和业务。

职位要求

- 1、计算机基础知识扎实，计算机、通信等相关专业优先；
- 2、2年及以上后端开发经验，熟练使用Go/Python/C++/Java等其中一种语言。

公司介绍

字节跳动成立于2012年3月，目前公司的产品和服务已覆盖全球150个国家和地区、75个语种，曾在40多个国家和地区排在应用商店总榜前列。

字节跳动在海外推出了多款有影响力的产品，包括综合资讯类的今日头条，视频类的抖音、TikTok、西瓜视频、抖音火山版、Vigo Video，以及教育产品、企业Sa

[查看全部](#)

再看华为的，好像Java架构师的要求啊，微服务，缓存，消息中间件，数据库。。。

职位描述

- 1、本科及以上学历，计算机相关专业，5年以上服务端研发经验；
- 2、深入理解计算机原理，有扎实的数据结构和算法基础；
- 3、深入理解linux系统及其原理，熟悉TCP/IP、HTTP协议以及网络编程；
- 4、良好的编码和文档习惯，注重代码风格，熟悉各种设计模式和原则，能持续的关注和优化自己做的项目；
- 5、熟悉Go、Java、C++、Python等至少一门语言，有Go开发经验者优先；
- 6、熟悉常用的互联网技术，包括但不限于MySQL、NoSQL、RPC、MQ、缓存技术、微服务架构等；
- 7、具有分布式系统设计和开发经验者优先。

这里不多看，自己看看吧，大多数你能知道的大公司都有用go语言尝试的新部门，新项目，市场占有率虽然比Java少，但是岗位实际上蛮多的。自己可以去BOSS上详细查查。

三.Go和Java微观对比

1.GoPath和Java的ClassPath

我们先来看看关于Java的classpath：

在我们的开发环境中，一个web程序(war包)有一个classpath,这个classpath在IDEA的开发工具中目录体现为src目录和resource目录，实际上在真正的war包中他定位的是指WEB-INF下的classes文件夹下的资源(比如class文件)。

我们编译后的文件都放在classpath(类路径)下。我们多个项目程序会有多个classpath目录。

在Go语言中，GoPath在同一系统上的同一用户，一般规定只有一个，无论这个用户创建多少个go项目，都只有一个GoPath,并且这些项目都放在GoPath下的src目录下。

GoPath下有三个目录：

- 1.bin （用于存放项目编译后的可执行文件）
- 2.pkg （用于存放类库文件，比如.a结尾的包模块）
- 3.src （用于存放项目代码源文件）

注意：当我们在windows上开发Go程序时，需要新建一个文件夹(文件夹名任意)作为GOPATH的文件目录，在其中新建三个文件夹分别是：bin, pkg, src。如果是在集成开发工具上开发的话，需要在设置中把GOPATH路径设置为你自定义的那个文件夹，之后产生的文件和相关内容都会在其中。

如果是在linux上想跑测试开发执行go程序，需要在/etc/profile添加名为GOPATH的环境变量，目录设置好自己新建的。

例如：全局用户设置GOPATH环境变量

```
vi /etc/profile
#添加如下 目录可以灵活修改
export GOPATH=/pub/go/gopath
//立即刷新环境变量生效
source /etc/profile
```

单用户设置GOPATH环境变量

```
vi ~/.bash_profile

#添加如下 目录可以自己灵活修改
export GOPATH=/home/user/local/soft/go/gopath
//立即刷新环境变量生效
source vi ~/.bash_profile
```

注意：这是在linux上开发go程序才需要的，如果只是生产运行程序的话是不需要任何东西的，直接运行二进制可执行程序包即可，他所有的依赖全部打进包中了。

如果是在windows下的cmd，dos窗口运行相关的go命令和程序，则需要在windows的【此电脑】-->【右键】-->【属性】-->【高级系统设置】-->【环境变量】-【新建一个系统变量】-->【变量名为GOPATH，路径为你自己指定的自定义文件夹】（如果是在IDEA中开发，不需要在此配置环境变量，只需要在IDEA中配置好GOPATH的目录设置即可）

2.Go的开发环境搭建

（配置环境变量GOPATH参考上一节内容）

我们要开发Go的程序，需要如下两样东西：

1.Go SDK

GO中文社区SDK下载地址：<https://studygolang.com/dl>

go1.14(最新的)

我们用1.14版就可以，因为1.13后才完全支持Module功能。

有两种安装模式，一种是压缩包解压的方式，一种是图形化安装。

推荐使用windows图形安装傻瓜式安装，windows图形安装下载这个

<https://studygolang.com/dl/golang/go1.14.6.windows-amd64.msi>

linux安装如下：

后续补上。。。

2. Go的集成软件开发环境

参考三(4)中的go集成开发环境选择。

3.Go与Java的文件结构对比

1).go文件结构模板

```
//主程序必须是写成main包名
package main

//导入别的类库
import "fmt"

//全局常量定义
const num = 10

//全局变量定义
var name string = "li_ming"
```



```
//类型定义
type P struct {

}

//初始化函数
func init() {

}

//main函数:程序入口
func main() {
    fmt.Printf("Hello world!!!");
}
```

2).Java文件结构

```
//包名
package my_package;

//导入包中的类
import java.io.*;

public class MainTest{
    //main方法:程序入口
    public void static main(String[] args) {

    }
}

//people类
class People {
    //成员变量
    public String name;
    public int age;

    //成员方法
    public void doSomething() {

    }
}
```

4.Go与Java的集成开发环境

1).Go的集成开发环境

最常用的有三种:

Visual Studio Code(VS Code) 微软开发的一款Go语言开发工具。

LiteIDE 是国人开发的Go语言开发工具。

GoLand 这个非常好用, 和Java中的IDEA是一家公司。(推荐使用)

2).Java的集成开发环境

MyEclipse,Eclipse(已过时)。

IntelliJ IDEA(大多数用这个)。

5.Go和Java常用包的对比

Go中文API文档地址:

<https://studygolang.com/pkgdoc>

	Go	Java
IO流操作:	bufio/os	java.lang.io
字符串操作:	strings	java.lang.String
容器	container(heap/list/ring)	java.lang.Collection
锁	sync	juc
时间	time	
java.time/java.lang.Date		
算数操作	math	java.math
底层Unsafe	unsafe	unsafe类

6.Go的常用基础数据类型和Java的基础数据类型对比

1).go中的常用基础数据类型有:

1.布尔型: 关键字【bool】: true false

2.有符号整形: 头一位是代表正负

int	默认整形	4或8字节	32位或64位
int8		1字节	8位
int16		2字节	16位
int32		4字节	32位
int64		8字节	64位

【int是32还是64位取决于操作系统的位数, 现在电脑一般都是64位的了, 所以一般都是64位】

3.无符号整形

uint	4或8字节	32位或64位
uint8	1字节	8位
uint16	2字节	16位
uint32	4字节	32位
uint64	8字节	64位

4.浮点型

注: go语言没有float类型, 只有float32和float64。

float32	32位浮点数
float64	64位浮点数

5.字符串

string

6. byte 等同uint8, 只是类似于一个别名的东西

rune 等同int32 只是一个别名, 强调表示编码概念对应的数字

2).go中派生数据类型有：

注：这里简单列举一下

指针	Pointer
数组	Array[]
结构体	struct
进程管道：	channel
函数	func
切片	slice
接口	interface
哈希	map

3).Java中的基础数据类型

```
byte

short

int

long

float

double

boolean

char
```

7.Go和Java的变量对比

1).go的变量

```
package main

import(
    //包含print函数
    "fmt"
)

func main() {
    //var 变量名 变量类型 = 变量值
    var name string = "li_ming"
    //方法内部可以直接使用 【 变量名 := 变量值 】 赋值，方法外不可以
    name2:="xiao_hong"
    fmt.Println("name = ",name)
    fmt.Println("name2 = ",name2)
}
```

2).Java的变量

```
public class MyTest {
    public static void main(String[] args) {
        //变量类型  变量名 = 变量值
        String name = "li_ming";
        int i = 10;
        System.out.println("name =" + name);
        System.out.println("i =" + i);
    }
}
```

8.Go和Java的常量对比

1).go的常量

go中的常量和java中的常量含义有一个本质的区别：
go中的常量是指在编译期间就能确定的量(数据)，
而java中的常量是指被赋值一次后就不能修改的量(数据)。
所以两者不一样，因为Java中的常量也是JVM跑起来后赋值的，只不过不允许更改；
go的常量在编译后就确实是什么数值了。

```
package main

import(
    //包含print函数
    "fmt"
)

func main() {
    //const 常量名 常量类型 = 常量值    显示推断类型
    const name string = "const_li_ming"
    //隐式推断类型
    const name2 ="const_xiao_hong"
    fmt.Println("name = ", name)
    fmt.Println("name2 = ", name2)
}
```

2).Java的常量

```
public class MyTest {
    //【访问修饰符】 【静态修饰符】final修饰符    常量类型    常量名 = 常量值；
    public static final String TAG = "A";    //一般设置为static静态
    public static void main(String[] args) {
        System.out.println("tag= " + TAG);
    }
}
```

9.Go与Java的赋值对比

1).go的赋值

Go方法内的赋值符号可以用 `:=` ,也可以用 `=`,方法外只能用 `=` 。
例如:

```
package main

import(
    //包含print函数
    "fmt"
)

//方法外只能用 = 赋值
var my_name string = "my_name"
var my_name2 = "my_name2"
//my_name3:="my_name3"    不在方法内，错误

func main() {
    fmt.Println("name = ",my_name)
    fmt.Println("name2 = ",my_name2)
}
```

Go支持多变量同时赋值:

```
package main

import(
    //包含print函数
    "fmt"
)

func main() {
    //多变量同时赋值
    var name,name2 = "li_ming","xiao_hong"
    fmt.Println("name = ",name)
    fmt.Println("name2 = ",name2)
}
```

Go的丢弃赋值

```

package main

import(
    //包含println函数
    "fmt"
)

func main() {
    //丢弃赋值    把 1和2丢弃 只取3
    //在必须一次取两个以上的值的场景下，又不想要其中一个值的时候使用，比如从map中取key,value
    var _,_,num = 1,2,3
    fmt.Println("num = ",num)
}

```

2).java的赋值

```

public class MyTest {
    public static void main(String[] args) {
        //直接用 = 赋值
        String name = "li_ming";
        int i = 10;
        System.out.println("name =" + name);
        System.out.println("i =" + i);
    }
}

```

10.Go与Java的注释

Go中的注释写法和Java中的基本一样。

例如：

//单行注释，两者相同

/*

Go的多行注释

*/

/**

Java多行注释

*/

11.Go和Java的访问权限设置区别

首先我们回忆一下，Java的权限访问修饰符有哪些？

public 全局可见

protected 继承相关的类可见

default 同包可见

private 私有的，本类可见

关于Java中的访问权限修饰符，是用于修饰变量，方法，类的，被修饰的对象被不同的访问权限修饰符修饰后，其它程序代码要想访问它，必须在规定的访问范围内才可以，比如同包，同类，父子类，全局均可访问。

那么，Go中的访问权限设置又有什么区别呢？

要理解这个问题，首先我们要来看下一个Go程序的程序文件组织结构是什么样子的？

一个可运行的编译后的Go程序，必须有一个入口，程序从入口开始执行，这个入口必须是main包，并且从main包的main函数开始执行。

但是，为了开发的效率和管理开发任务的协调简单化，对于代码质量的可复用，可扩展等特性的要求，我们一般采用面向对象的，文件分模块式的开发。

比如，我是一个游戏程序，我的main函数启动后，首先要启动UI界面，那么关于UI界面相关的代码我们一般会专门分出一个模块去开发，然后这个模块有很多个程序文件，这里关于UI模块比如有3个文件，a.go,b.go,c.go，那么我们在实际当中会建一个以ui为名的包文件夹，然后把a.go,b.go,c.go全部放到ui这个包文件夹下，然后因为这个包没有main包，没有main函数，所以它打出来的一个程序文件就是以.a结尾的工具包，类似于Java中的jar包，工具包文件名为 ui.a。

参考如下：

```
----com.mashibing.mygame.ui
    -----a.go
    -----b.go
    -----c.go
```

a.go文件如下示例：

```
//这里的ui，也就是package后面的名称尽量和包文件夹的名称一致，不一致也可以
package ui

//相关方法和业务

func main() {

}

//启动游戏UI
func StartGameUI() {

}
```

这里需要注意一个点，在程序中的 package后面的 ui包名可以和文件夹com.mashibing.mygame.ui中最后一层的ui文件夹名称不一致，

我们一般按规范写是要求写一致的，不一致时的区别如下：

我们把ui.a打包完毕后，我们就可以在别的程序中用import导入这个包模块，然后使用其中的内容了。

上面两个ui不同之处在于，在我们import 的代码后面，需要写的模块名称是在 \${gopath}/src/下的文件夹名，也就是com.mashibing.mygame.ui中的ui。

例如：

```
//游戏主程序
package main

//这里的ui是com.mashibing.mygame.ui的最后一层文件夹名
import "ui"

//相关方法和业务

func main() {
    //这里的ui不是文件夹名，而是之前a.go程序中package后面写的包名
    ui.StartGameUI()
}
```

› 此电脑 › 系统 (C:) › Go › src

名称	修改日期
archive	2020/6/29 8:35
bufio	2020/6/29 8:35
builtin	2020/6/29 8:35
bytes	2020/6/29 8:35
cmd	2020/6/29 8:35
compress	2020/6/29 8:35
container	2020/6/29 8:35
context	2020/6/29 8:35
crypto	2020/6/29 8:35
database	2020/6/29 8:35
debug	2020/6/29 8:35
encoding	2020/6/29 8:35
errors	2020/6/29 8:35
expvar	2020/6/29 8:35
flag	2020/6/29 8:35
fmt	2020/6/29 8:35
go	2020/6/29 8:35
hash	2020/6/29 8:35
html	2020/6/29 8:35
image	2020/6/29 8:35


```
File Edit View Help
print.go x
1 // Copyright 2009 The Go Authors. All rights reserved.
2 // Use of this source code is governed by a BSD-style
3 // license that can be found in the LICENSE file.
4
5 package fmt
6
7 import (
8     "errors"
9     "io"
10    "os"
11    "reflect"
12    "sync"
13    "unicode/utf8"
14 )
15
16 // Strings for use with buffer.WriteString.
17 // This is less overhead than using buffer.Write with byte arrays.
18 const (
19     commaSpaceString = ", "
20     nilAngleString   = "<nil>"
21     nilParenString   = "(nil)"
22     nilString        = "nil"
23     mapString        = "map["
24     percentBandString = "%!"
```

接下来进入主题，我们的go语言关于访问修饰符的是指的限制什么权限，以及如何实现？

我们之前可以看出来，实战中的go程序是有一个main程序import很多其它包模块，每个模块实现对应的功能，最后统一在main程序中组合来完成整个软件程序，那么有一些其它模块的函数和变量，我只想在本程序文件中调用，不想被其它程序import能调用到，如何实现？

import后是否能调用对应包中的对象(变量，结构体，函数之类的)就是go关于访问权限的定义，import后，可以访问，说明是开启了访问权限，不可以访问，是说明关闭了其它程序访问的权限。

在go中，为了遵循实现简洁，快速的原则，用默认规范来规定访问权限设置。

默认规范是：某种类型（包括变量，结构体，函数，类型等）的名称定义首字母大写就是在其它包可以访问，首字母非大写，就是只能在自己的程序中访问。

这样我们就能理解为什么导入fmt包后，他的Printf函数的首字母P是大写的。

参照如下代码：

```
package ui

import "fmt"

func main() {
    //这里的P是大写
    //所有调用别的包下的函数，都是首字母大写
    fmt.Printf("aa")
}

//这里的Person的首字母P也是表示外部程序导入该包后可以使用此Person类
type Person struct{
```

```
}
//这里的D同上
var Data string = "li_ming"
```

12.Go与Java程序文件的后缀名对比

Java的编译文件是.class结尾,多个.class打成的一个可执行文件是.jar结尾,.jar不能直接在windows和linux上执行,得用java命令在JVM中执行。

Go语言的程序文件后缀为.go,有main包main函数的,.go文件打包成二进制对应操作系统的可执行程序,如windows上的.exe结尾的可执行程序。

Java的类库会以.jar结尾,Go语言非main包没有main函数的程序编译打包会打成一个类库,以.a结尾,也就是说Go语言的类库以.a结尾。

Go的类库如下:

```
包名.a
my_util.a
```

注: my_util是最顶层文件夹名,里面包含着一个个程序文件。

13.Go与Java选择结构的对比

1).if

Go中的if和Java中的if使用相同,只不过是把小括号给去掉了。

示例1:

```
package main

import (
    "fmt"
)

func main() {
    /*
        单分支结构语法格式如下:
        if 条件判断 {
            //代码块
        }
    */

    var num int

    fmt.Printf("请输入数字")
    fmt.Scan(&num)

    if num > 10 {
        fmt.Println("您输入的数字大于10")
    }
}
```

示例2:

```

package main

import (
    "fmt"
)

func main() {
    /*
        if else分支结构语法格式如下：
        if 条件判断 {
            //代码块
        } else {
            //代码块2
        }
    */

    var num int

    fmt.Printf("请输入数字")
    fmt.Scan(&num)

    if num > 10 {
        fmt.Println("您输入的数字大于10")
    } else {
        fmt.Println("您输入的数字不大于10")
    }
}

```

示例3:

```

package main

import (
    "fmt"
)

func main() {
    /*
        if else分支结构语法格式如下：
        if 条件判断 {
            //代码块
        } else if 条件判断{
            //代码块2
        } else {
            //代码块3
        }
    */

    var num int

    fmt.Printf("请输入数字")
    fmt.Scan(&num)

    if num > 10 {
        fmt.Println("您输入的数字大于10")
    } else if num == 10{

```

```

        fmt.Println("您输入的数字等于10")
    } else {
        fmt.Println("您输入的数字小于10")
    }
}

```

2).switch

示例1:

```

package main

import (
    "fmt"
)

func main() {
    var a = "li_ming"
    switch a {
    case "li_ming":
        fmt.Println("Hello!LiMing")
    case "xiao_hong":
        fmt.Println("Hello!XiaoHong")
    default:
        fmt.Println("No!")
    }
}

```

示例2：一分支多值

```

package main

import (
    "fmt"
)

func main() {

    var name = "li_ming"
    var name2 = "xiao_hong"
    switch name {
    //li_ming或xiao_hong 均进入此
    case "li_ming", "xiao_hong":
        fmt.Println("li_ming and xiao_hong")
    }

    switch name2 {
    case "li_ming", "xiao_hong":
        fmt.Println("li_ming and xiao_hong")
    }
}g

```

示例3: switch表达式

```

package main

import (
    "fmt"
)

func main() {
    var num int = 11
    switch{
        case num > 10 && num < 20:
            fmt.Println(num)
    }
}

```

示例4: fallthrough下面的case全部执行

```

package main

import (
    "fmt"
)

func main() {
    var num = 11
    switch {
        case num == 11:
            fmt.Println("==11")
            fallthrough
        case num < 10:
            fmt.Println("<12")
    }
}

```

14.Go与Java循环结构的对比

1).for循环

示例1: 省略小括号

```

package main

import (
    "fmt"
)

func main() {
    for i := 1; i < 10; i++ {
        fmt.Println(i)
    }
}

```

示例2: 和while相同, break,continue同java

```

package main

```

```

import (
    "fmt"
)

func main() {
    i := 0
    //省略另外两项，相当于java中的while
    for i < 3 {
        i++
    }
    //break用法相同
    for i == 3 {
        fmt.Println(i)
        break
    }
}

```

示例3: 死循环，三项均省略

```

package main

func main() {
    for {

    }

    for true {

    }

}

```

示例4: 嵌套循环和java也一样，不演示了

示例5: range循环

```

package main

import "fmt"

func main() {
    var data [10]int = [10]int{1,2,3,4,5,6,7,8,9,10}
    for i, num := range data {
        fmt.Println(i,num)
    }
}

```

2).goto

```

package main

import "fmt"

func main() {

```

```
//goto可以用在任何地方，但是不能跨函数使用
fmt.Println("start")

//go to的作用是跳转，中间的语句不执行，无条件跳转
goto my_location //goto是关键字， my_location可以自定义，他叫标签

fmt.Println("over")
my_location:
fmt.Println("location")
```

15.Go与Java的数组对比

1) go的一维数组

```
var 数组名 [数组长度]数组类型 = [数组长度]数组类型{元素1, 元素2...}
```

示例1:

```
package main

import "fmt"
//全局
var my_arr [6]int
var my_arr_1 [3]int = [3]int{1,2,3}
func main() {
    //方法内:
    this_arr := [2]int{1, 2}
    fmt.Println(my_arr)
    fmt.Println(my_arr_1)
    fmt.Println(this_arr)
}
```

2) 二维数组

```
package main

import "fmt"
//全局
var my_arr [4][6]int
var my_arr_1 [2][3]int = [...] [3]int{{1, 2, 3}, {4, 5, 6}}
func main() {
    //方法内:
    this_arr := [2][3]int{{1, 2, 3}, {8, 8, 8}}
    // 第 2 纬度不能用 "...".
    this_arr2 := [...] [2]int{{1, 1}, {2, 2}, {3, 3}}
    fmt.Println(my_arr)
    fmt.Println(my_arr_1)
    fmt.Println(this_arr)
    fmt.Println(this_arr2)
}
```

16.Go有指针概念，Java没有指针概念

Go中有指针的概念，Java中没有指针的概念。
指针简单的说就是存储一个【变量地址】的【变量】。

Go中使用指针的方法

//*+变量类型 = 对应变量的指针类型，&+变量名 = 获取变量引用地址

var 指针变量名 *指针变量类型 = &变量名

例如：

```
var my_point *int = &num
```

//通过&+指针变量 = 修改原来的变量真实值

&指针变量名 = 修改的变量值

例如：

```
&my_point = 100;
```

示例：

```
package main

import "fmt"

func main() {
    // 声明实际变量
    var name string="li_ming"
    // 声明指针变量
    var name_point *string
    // 指针变量的存储地址
    name_point = &name

    //直接访问变量地址
    fmt.Println("name 变量的地址是:", &name )

    // 指针变量的存储地址
    fmt.Println("name_point变量储存的指针地址:", name_point )

    // 使用指针访问值
    fmt.Println("*name_point 变量的值:", *name_point )
}
```

输出结果：

```
name 变量的地址是：0x10ae40f0
name_point变量储存的指针地址：0x10ae40f0
*name_point 变量的值：li_ming
```

17.Go语言的中new,make和Java中的new对象有什么区别？

首先，Java中的new关键字代表创建关于某一个类的一个新的对象。

如：

```
List list = new ArrayList();
```


Go中的创建一个struct结构体的对象，是不需要用new关键字的，参考【20】中有代码示例讲解如何创建结构体对象。

Go中新new的概念是和内存相关的，我们可以通过new来为基础数据类型申请一块内存地址空间，然后把这块内存地址空间赋值给

一个指针变量上。（new主要就是为基础数据类型申请内存空间的，当我们需要一个基础数据类型的指针变量，并且在初始化这个基础指针变量时，不能确定他的初始值，此时我们才需要用new去内存中申请一块空间，并把这块空间绑定到对应的指针上，之后可以用该指针为这块内存空间写值。new关键字在实际开发中很少使用，和java很多处用new的情况大不相同）

参考如下示例代码：

```
package main

import "fmt"

func main() {
    var num *int
    //此处num是nil
    fmt.Println(num)
    //此处会报空指针异常，因为num为nil,没有申请内存空间，所以不能为nil赋值
    *num = 1
    fmt.Println(*num)
}
```

改为如下代码即可：

```
package main

import "fmt"

func main() {
    //在内存中申请一块地址，并把内存地址存入num
    var num = new(int)
    //此处num的值是申请出来的内存空间地址值，一个十六进制的数字
    fmt.Println(num)
    //正常
    *num = 1
    fmt.Println(*num)
}
```

接下来我们来看一个go中的make是做什么用的？

go中的make是用来创建slice(切片)，map(映射表)，chan(线程通信管道)这三个类型的对象的，返回的就是对应类型的对象，他的作用就相当于Java中新new一个ArrayList，new一个HashMap时候的new的作用，只不过是go语法规定用make来创建slice(切片)，map(映射表)，chan(线程通信管道)。

示例代码如下：

```
package main

import "fmt"

func main() {
```

```

//make只能为map,channel,slice申请分配内存，只有这三种，没有第四种
//所有通过make创建的这三种类型都是引用类型，传递参数时虽然是引用值传递，
//但是对方法内引用变量参数的修改可以影响到外部的引用变量
//1.通过make创建map对象 如下代码类似于Java中 Map<String,Integer> myMap = new
HashMap<>();
//在这里make就是申请分配map的内存，和java中创建map的new一样
myMap := make(map[string]int)
myMap["li_ming"] = 20

//2.通过make创建channel,make函数内可以有一个参数，也可以有两个参数，有两个参数时第二个参
数
//为通道的缓存队列的长度
//2.1) 只有一个参数，通道的缓存队列长度此时为0，也就是无缓存。
//创建一个传输int类型数据的通道
myChan := make(chan int)
fmt.Println(myChan)
//2.2) 有两个参数，第二个参数2代表此时代表缓存队列的长度为2
//创建一个传输int类型数据的通道,缓存为2
mychan2 := make(chan int,2)
fmt.Println(mychan2)
//此处暂时不做通道缓存队列数多少有何区别的讲解

//3.通过make创建slice切片
//有两种方式，一种是两个参数，一种是三个参数
//我们只有在创建一个空的切片时才会使用make
//如果通过一个已有的数组创建切片往往是下面的形式
//创建一个底层数组
myArr := []int{1,2,3,4,5}
//如果通过一个数组创建切片，往往是用 原始数组变量名[切片起始位置:切片结束位置] 创建一个切片
mySlice1 := myArr[2:4]
fmt.Println(mySlice1)
//我们如果是想创建一个空的slice,则用make创建切片
//如下形式 make(int[],num1,num2)
//num1 = 切片的长度(默认分配内存空间的元素个数)
//num2 = 切片的容量(解释：底层数组的长度/切片的容量，超过底层数组长度append新元素时会创建
一个新的底层数组，
//不超过则会使用原来的底层数组)

//代表底层数组的长度是4,默认给底层数组的前两个元素分配内存空间
//切片指向前两个元素的地址，如果append新元素，在元素数小于4时都会
//在原来的底层数组的最后一个元素新分配空间和赋值，
//append超过4个元素时，因为原数组大小不可变，也存储不下了，
//所以会新创建一个新的底层数组，切片指向新的底层数组
mySliceEmpty := make([]int,2,4)
fmt.Println(mySliceEmpty)

//两个参数，代表切片的长度和切片的容量(底层数组长度)均为第二个参数那个值
mySliceEmpty2 := make([]int,5)
fmt.Println(mySliceEmpty2)
}

```

18.Go相关的数据容器和Java的集合框架对比

Go中有的数据结构：数组，切片，map，双向链表，环形链表，堆
Go自己的类库中没有set,没有集合(List)，但是第三方库有实现。
Java中有： Map,Set,List,Queue,Stack,数组
Java中没有切片的概念。
Go中的数组打印格式是[1,2,3,4,5]
Go中的切片打印格式是[[1,2,3]]
Go中切片的概念：切片是数组的一个子集，就是数组截取某一段。
Go的map和Java的map大致相同

19.Go中的函数，Go的方法和Java中的方法对比

1).Go中的函数定义

Go中返回值可以有多个，不像Java中多个值得封装到实体或map返回
//注：【】内的返回值可不写，无返回值直接把返回值部分全部去掉即可。
func 函数名(变量1 变量类型, 变量2 变量2类型...)【(返回值1 类型1, 返回值2 类型2...)】 {
//注意：这个方法的右中括号必须和func写在同一行才行，否则报错，不能按c语言中的换行写

示例1：

```
package main

import "fmt"

func main() {
    //定义局部变量
    var a int = 100
    var b int = 200
    var result int

    //调用函数并返回最大值
    result = max(a, b)

    fmt.Println( "最大值是 :", result )
}

/* 函数返回两个数的最大值 */
func max(num1, num2 int) int {
    /* 定义局部变量 */
    var result int

    if (num1 > num2) {
        result = num1
    } else {
        result = num2
    }
    return result
}
```

示例2：返回多个值

```

package main

import "fmt"

func main() {
    a, b := swap("li_ming", "xiao_hong")
    fmt.Println(a, b)
}

func swap(x, y string) (string, string) {
    //返回多个值
    return y, x
}

```

注意点：函数的参数：基础类型是按值传递，复杂类型是按引用传递

示例3：函数的参数：变长参数传递

```

package main

import "fmt"

func main() {
    manyArgs(1,2,"2","3","4")
    manyArgs(1,2,"5","5","5")
    dataStr := []string{"11","11","11"}
    //传数组也可以，加三个点
    manyArgs(1,2,dataStr...)
}

//可变参数必须放在最后面
func manyArgs(a int,b int ,str ...string ){
    for i,s := range str {
        fmt.Println(i,s)
    }
}

```

注意点：函数的返回值：如果有返回值，返回值的类型必须写，返回值得变量名根据情况可写可不写。

示例4：defer：推迟执行(类似于java中的finally)

```

package main

import "fmt"

func main() {
    testMyFunc();
}

func testDefer1() {
    fmt.Println("print defer1")
}

func testDefer2() {
    fmt.Println("print defer2")
}

```

```

}

func testMyFunc() {
    //defer会在方法返回前执行，有点像java中的finally
    //defer写在任意位置均可，多个defer的话按照逆序依次执行
    defer testDefer2()
    defer testDefer1()
    fmt.Println("print my func")
}

```

示例5：丢弃返回值

```

package main

import "fmt"

func main() {
    //方式一丢弃：丢弃num1和str
    _, num2, _ := testFun(1, 2, "3");
    fmt.Println(num2)
    //方式二丢弃：
    _, num3, _ := testFun(1, 3, "4");
    fmt.Println(num3)
}

func testFun(num1, num2 int, str string) (n1 int, n2 int, s1 string){
    n1 = num1
    n2 = num2
    s1 = str
    return
}

func testFun2(num1, num2 int, str string) (int, int, string){
    return num1, num2, str
}

```

2).Java中的方法定义

```

访问修饰符    返回值类型    方法名(参数1类型    参数1, 参数2类型 参数2...) {

    return 返回值;

}

```

示例：

```

public Integer doSomething(String name,Integer age) {

    return 20;

}

```

20.Go的内置函数和Java的默认导入包java.lang.*

为了在Java中快速开发，Java语言的创造者把一些常用的类和接口都放到到java.lang包下，lang包下的特点就是不用写import语句导入包就可以用里面的程序代码。

Go中也有类似的功能，叫做Go的内置函数，Go的内置函数是指不用导入任何包，直接就可以通过函数名进行调用的函数。

Go中的内置函数有：

close	关闭channel
len	求长度
make	创建slice,map,chan对象
append	追加元素到切片(slice)中
panic	抛出异常，终止程序
recover	尝试恢复异常，必须写在defer相关的代码块中

参考示例代码1：

```
package main

import "fmt"

func main() {

    array := [5]int{1,2,3,4,5}
    str := "myName"
    //获取字符串长度
    fmt.Println(len(str))
    //获取数组长度
    fmt.Println(len(array))
    //获取切片长度
    fmt.Println(len(array[1:]))

    //make创建channel示例
    intChan := make(chan int,1)
    //make创建map示例
    myMap := make(map[string]interface{})
    //make创建切片
    mySlice := make([]int,5,10)

    fmt.Println(intChan)
    fmt.Println(myMap)
    fmt.Println(mySlice)

    //关闭管道
    close(intChan)
    //为切片添加元素
    array2 := append(array[:],6)
    //输出
```

```

    fmt.Println(array2)

    //new案例
    num := new(int)
    fmt.Println(num)

}

```

参考示例代码2: panic和recover的使用

他们用于抛出异常和尝试捕获恢复异常

```

func func1() {
    fmt.Println("1")
}

func func2() {
    // 刚刚打开某资源
    defer func() {
        err := recover()
        fmt.Println(err)
        fmt.Println("释放资源..")
    }()
    panic("抛出异常")
    fmt.Println("2")
}

func func3() {
    fmt.Println("3")
}

func main() {
    func1()
    func2()
    func3()
}

```

Java中的java.lang包下具体有什么在这里就不赘述了, 请参考JavaAPI文档:

JavaAPI文档导航: <https://www.oracle.com/cn/java/technologies/java-se-api-doc.html>

21.Go的标准格式化输出库fmt和java的输出打印库对比

Java的标准输出流工具类是java.lang包下的System类, 具体是其静态成员变量PrintStream类。

他有静态三个成员变量:

分别是PrintStream类型的out,in,err

我们常见System.out.println(),实际上调用的就是PrintStream类对象的println方法。

Go中的格式化输出输入库是fmt模块。

fmt在Go中提供了输入和输出的功能, 类型Java中的Scanner和PrintStream(println)。

它的使用方法如下：

```
Print:    原样输出到控制台，不做格式控制。
Println:  输出到控制台并换行
Printf :  格式化输出(按特定标识符指定格式替换)
Sprintf:  格式化字符串并把字符串返回，不输出，有点类似于Java中的拼接字符串然后返回。
Fprintf:  来格式化并输出到 io.writers 而不是 os.Stdout
```

详细占位符号如下：

代码示例如下：

22.Go的面向对象相关知识

1.封装属性(结构体)

Go中有一个数据类型是Struct,它在面向对象的概念中相当于Java的类，可以封装属性和封装方法，首先看封装属性如下示例：

```
package main

import "fmt"

//示例
type People struct {
    name string
    age  int
    sex  bool
}

func main(){

    //示例1:
    var l1 People
    l1.name = "li_ming"
    l1.age = 22
    l1.sex = false
    //li_ming
    fmt.Println(l1.name)

    //示例2
    var l2 *People = new(People)
    l2.name = "xiao_hong"
    l2.age = 33
    l2.sex = true
    //xiao_hong xiao_hong
    fmt.Println(l2.name, (*l2).name)

    //示例3:
    var l3 *People = &People{ name:"li_Ming",age:25,sex:true}
    //li_Ming li_Ming
    fmt.Println(l3.name, (*l3).name)
}
```


2.封装方法(方法接收器)

如果想为某个Struct类型添加一个方法，参考如下说明和代码：

go的方法和java中的方法对比，go的函数和go方法的不同

Go中的函数是不需要用结构体的对象来调用的，可以直接调用

Go中的方法是必须用一个具体的结构体对象来调用的，有点像java的某个类的对象调用其方法

我们可以把指定的函数绑定到对应的结构体上，使该函数成为这个结构体的方法，然后这个结构体的对象就可以通过.来调用这个方法了

绑定的形式是：在func和方法名之间写一个(当前对象变量名 当前结构体类型)，这个叫方法的接受器，其中当前对象的变量名就是当前结构体调用该方法的对象的引用，相当于java中的this对象。

参考如下示例为Student学生添加一个learn学习的方法

```
package main

import "fmt"

type Student struct {
    num int //学号
    name string //姓名
    class int //班级
    sex bool //性别
}

//给Student添加一个方法
//这里的(stu Student)中的stu相当于java方法中的this对象
//stu是一个方法的接收器，接收是哪个对象调用了当方法
func (stu Student) learn() {
    fmt.Printf("%s学生正在学习", stu.name)
}

func main() {
    stu := Student{1, "li_ming", 10, true}
    stu.learn()
}
```

方法的接收器也可以是指针类型的

参考如下案例：

```
package main

import "fmt"

type Student struct {
    num int //学号
    name string //姓名
    class int //班级
    sex bool //性别
}

//这里方法的接收器也可以是指针类型
```

```
func (stu *Student) learn() {
    fmt.Printf("%s学生正在学习", stu.name)
}

func main() {
    //指针类型
    stu := &Student{1, "li_ming", 10, true}
    stu.learn()
}
```

注意有一种情况，当一个对象为nil空时，它调用方法时，接收器接受的对于自身的引用也是nil，需要我们先做一些健壮性的不为nil才做的判断处理。

3.Go的继承(结构体嵌入)

Go中可以用嵌入结构体实现类似于继承的功能：

参考如下代码示例：

```
package main

import "fmt"

//电脑
type Computer struct {
    screen string //电脑屏幕
    keyboard string //键盘
}

//计算方法
func (cp Computer) compute(num1, num2 int) int {
    return num1 + num2;
}

//笔记本电脑
type NotebookComputer struct {
    Computer
    wireless_network_adapter string //无线网卡
}

func main() {
    var cp1 NotebookComputer = NotebookComputer{}
    cp1.screen = "高清屏"
    cp1.keyboard = "防水键盘"
    cp1.wireless_network_adapter = "新一代无线网卡"
    fmt.Println(cp1)
    fmt.Println(cp1.compute(1, 2))
}
```

需要注意的是，Go中可以嵌入多个结构体，但是多个结构体不能有相同的方法，如果有参数和方法名完全相同的方法，在编译的时候就会报错。所以Go不存在嵌入多个结构体后，被嵌入的几个结构体有相同的方法，最后不知道选择执行哪个方法的情况，多个结构体方法相同时，直接编译就会报错。

参考如下示例：

```
package main
```

```

import "fmt"

func main() {
    man := Man{}
    fmt.Println(man)
    //下面的代码编译会报错
    //man.doEat()

}

type Man struct {
    FatherA
    FatherB
}

func (p FatherA) doEat() {
    fmt.Printf("FatherA eat")
}

func (t FatherB) doEat() {
    fmt.Printf("FatherB eat")
}

type FatherB struct {

}

type FatherA struct {
}

```

4.Go的多态(接口)

接下来我们讲Go中如何通过父类接口指向具体实现类对象，实现多态：

go语言中的接口是非侵入式接口。

java语言中的接口是侵入式接口。

侵入式接口是指需要显示的在类中写明实现哪些接口。

非侵入式接口是指不要显示的在类中写明要实现哪些接口，只需要方法名同名，参数一致即可。

参考如下代码示例：接口与多态

```

package main

import "fmt"

//动物接口
type Animal interface{
    eat()      //吃饭接口方法
    sleep()    //睡觉接口方法
}

//小猫
type Cat struct {
}

```

```

}
//小狗
type Dog struct {

}
//小猫吃方法
func (cat Cat) eat() {
    fmt.Println("小猫在吃饭")
}
//小猫睡方法
func (cat Cat) sleep(){
    fmt.Println("小猫在睡觉")
}
//小狗在吃饭
func (dog Dog) eat(){
    fmt.Println("小狗在吃饭")
}
//小狗在睡觉
func (dog Dog) sleep(){
    fmt.Println("小狗在睡觉")
}

func main() {
    var cat Animal = Cat{}
    var dog Animal = Dog{}
    cat.eat()
    cat.sleep()
    dog.eat()
    dog.sleep()
}

```

接口可以内嵌接口

参考如下代码示例:

```

package main
//内嵌接口
//学习接口，内嵌听和看学习接口
type Learn interface {
    LearnByHear
    LearnByLook
}
//通过听学习接口
type LearnByHear interface {
    hear()
}
//通过看学习
type LearnByLook interface {
    look()
}

```

###

23.Go语言中线程的实现和Java语言中线程的实现

go中的线程相关的概念是Goroutines(并发)，是使用go关键字开启。

Java中的线程是通过Thread类开启的。

在go语言中，一个线程就是一个Goroutines，主函数就是（主） main Goroutines。

使用go语句来开启一个新的Goroutines

比如：

普通方法执行

```
myFunction()
```

开启一个Goroutines来执行方法

```
go myFunction()
```

java中是

```
new Thread()->{  
    //新线程逻辑代码  
}.start();
```

参考下面的代码示例：

```
package main  
  
import (  
    "fmt"  
)  
  
//并发开启新线程goroutine测试  
  
//我的方法  
func myFunction() {  
    fmt.Println("Hello!!!")  
}  
//并发执行方法  
func goroutineTestFunc() {  
    fmt.Println("Hello!!! Start Goroutine!!!")  
}  
  
func main() {  
    /*  
    myFunction()  
    //go goroutineTestFunc()  
    //此时因为主线程有时候结束的快，goroutineTestFunc方法得不到输出，由此可以看出是开启了新的  
    线程。  
    */  
    //打开第二段执行  
    /*  
    go goroutineTestFunc()  
    time.Sleep(10*time.Second)//睡一段时间 10秒
```

```

    myFunction()
    */
}

```

线程间的通信：

java线程间通信有很多多种方式：

比如最原始的 wait/notify

到使用juc下高并发线程同步容器，同步队列

到CountDownLatch等一系列工具类

.....

甚至是分布式系统不同机器之间的消息中间件，单机的disruptor等等。

Go语言不同，线程间主要的通信方式是Channel。

Channel是实现go语言多个线程（goroutines）之间通信的一个机制。

Channel是一个线程间传输数据的管道，创建Channel必须声明管道内的数据类型是什么

下面我们创建一个传输int类型数据的Channel

代码示例：

```

package main

import "fmt"

func main() {
    ch := make(chan int)
    fmt.Println(ch)
}

```

channel是引用类型，函数传参数时是引用传递而不是值拷贝的传递。

channel的空值和别的应用类型一样是nil。

==可以比较两个Channel之间传输的数据类型是否相等。

channel是一个管道，他可以收数据和发数据。

具体参照下面代码示例：

```

package main

import (
    "fmt"
    "time"
)
//channel发送数据和接受数据用 <-表示,是发送还是接受取决于chan在 <-左边还是右边
//创建一个传输字符串数据类型的管道
var chanStr = make(chan string)
func main() {
    fmt.Println("main goroutine print Hello ")
}

```

```

//默认channel是没有缓存的，阻塞的，也就是说，发送端发送后直到接受端接受到才会施放阻塞往下走。
//同样接收端如果先开启，直到接收到数据才会停止阻塞往下走
//开启新线程发送数据
go startNewGoroutineOne()
//从管道中接收读取数据
go startNewGoroutineTwo()
//主线程等待，要不直接结束了
time.Sleep(100*time.Second)
}

func startNewGoroutineOne() {
    fmt.Println("send channel print Hello ")
    //管道发送数据
    chanStr <- "Hello!!!"
}

func startNewGoroutineTwo(){
    fmt.Println("receive channel print Hello ")
    strVar := <-chanStr
    fmt.Println(strVar)
}

```

无缓存的channel可以起到一个多线程间线程数据同步锁安全的作用。

缓存的channel创建方式是

make(chan string,缓存个数)

缓存个数是指直到多个数据没有消费或者接受后才进行阻塞。

类似于java中的synchronized和lock

可以保证多线程并发下的数据一致性问题。

首先我们看一个线程不安全的代码示例：

```

package main

import (
    "fmt"
    "time"
)

//多线程并发下的不安全问题
//金额
var moneyA int =1000
//添加金额
func subtractMoney(subMoney int) {
    time.Sleep(3*time.Second)
    moneyA-=subMoney
}

//查询金额
func getMoney() int {
    return moneyA;
}

```

```

func main() {

    //添加查询金额
    go func() {
        if(getMoney()>200) {
            subtractMoney(200)
            fmt.Printf("200元扣款成功，剩下：%d元\n",getMoney())
        }
    }()

    //添加查询金额
    go func() {
        if(getMoney()>900) {
            subtractMoney(900)
            fmt.Printf("900元扣款成功，剩下：%d元\n",getMoney())
        }
    }()

    //正常逻辑，只够扣款一单，可以多线程环境下结果钱扣多了
    time.Sleep(5*time.Second)
    fmt.Println(getMoney())
}

```

缓存为1的channel可以作为锁使用：

示例代码如下：

```

package main

import (
    "fmt"
    "time"
)

//多线程并发下使用channel改造
//金额
var moneyA = 1000
//减少金额管道
var synchLock = make(chan int,1)

//添加金额
func subtractMoney(subMoney int) {
    time.Sleep(3*time.Second)
    moneyA-=subMoney
}

//查询金额
func getMoney() int {
    return moneyA;
}

func main() {

```



```

//添加查询金额
go func() {
    synchLock<-10
    if(getMoney()>200) {
        subtractMoney(200)
        fmt.Printf("200元扣款成功，剩下： %d元\n",getMoney())
    }
    <-synchLock
}()

//添加查询金额
go func() {
    synchLock<-10
    if(getMoney()>900) {
        subtractMoney(900)
        fmt.Printf("900元扣款成功，剩下： %d元\n",getMoney())
    }
    synchLock<-10
}()
//这样类似于java中的Lock锁，不会扣多
time.Sleep(5*time.Second)
fmt.Println(getMoney())
}

```

go也有互斥锁

类似于java中的Lock接口

参考如下示例代码：

```

package main

import (
    "fmt"
    "sync"
    "time"
)

//多线程并发下使用channel改造
//金额
var moneyA = 1000

var lock sync.Mutex;
//添加金额
func subtractMoney(subMoney int) {
    lock.Lock()
    time.Sleep(3*time.Second)
    moneyA-=subMoney
    lock.Unlock()
}

//查询金额
func getMoney() int {
    lock.Lock()
    result := moneyA
    lock.Unlock()
}

```

```

    return result;
}

func main() {
    //添加查询金额
    go func() {
        if(getMoney()>200) {
            subtractMoney(200)
            fmt.Printf("200元扣款成功，剩下: %d元\n",getMoney())
        }else {
            fmt.Println("余额不足，无法扣款")
        }
    }()

    //添加查询金额
    go func() {
        if(getMoney()>900) {
            subtractMoney(900)
            fmt.Printf("900元扣款成功，剩下: %d元\n",getMoney())
        }else {
            fmt.Println("余额不足，无法扣款")
        }
    }()

    //正常
    time.Sleep(5*time.Second)
    fmt.Println(getMoney())
}

```

24.Go中的反射与Java中的反射对比

整体概述：反射是一个通用的概念，是指在程序运行期间获取到变量或者对象，结构体的元信息，比如类型信息，并且能够取出其中变量的值，调用对应的方法。

首先我们先来回顾一下Java语言用到反射的场景有哪些？

- 1.比如说我们的方法参数不能确定是什么类型，是Object类型，我们就可以通过反射在运行期间获取其真实的类型，然后做对应的逻辑处理。
- 2.比如动态代理，我们需要在程序运行时，动态的加载一个类，创建一个类，使用一个类。
- 3.比如在想要强行破解获取程序中被private的成员。
- 4.Java的各种框架中用的非常多，框架中用反射来判断用户自定义的类是什么类型，然后做区别处理。

Go中的反射大概也是相同的，比如，go中有一个类型 interface,interface类型相当于Java中的Object类，当以interface作为参数类型时，可以给这个参数传递任意类型的变量。

例如示例1：

```

package main

import "fmt"

func main() {
    testAllType(1);
    testAllType("Go");
}

//interface{}代表任意类型
func testAllType (data interface{}){
    fmt.Println(data)
}

```

那么第一种应用场景就出现了，当我们在go中想实现一个函数/方法，这个函数/方法的参数类型在编写程序的时候不能确认，在运行时会有各种不同的类型传入这个通用的函数/方法中，我们需要对不同类型的参数做不同的处理，那么我们就得能获取到参数是什么类型的，然后根据这个类型信息做业务逻辑判断。

反射我们需要调用reflect包模块,使用reflect.TypeOf()可以获取参数的类型信息对象，再根据类型信息对象的kind方法，获取到具体类型，详细参考下面代码。

例如示例2：

```

package main

import (
    "fmt"
    "reflect"
)

func main() {
    handleType(1)
    handleType(true)
}

func handleType(data interface{}) {
    //获取类型对象
    d := reflect.TypeOf(data)
    //kind方法是获取类型
    fmt.Println(d.Kind())
    switch d.Kind() {
        case reflect.Invalid:
            //无效类型逻辑处理
            fmt.Println("无效类型")
        case reflect.Int, reflect.Int8, reflect.Int16, reflect.Int32, reflect.Int64:
            fmt.Println("整形")
        case reflect.Bool:
            fmt.Println("bool类型")
    }
}

```

因为传入进来的都是interface类型，所以我们需要用的时候要区分类型，然后取出其中真正类型的值。

反射取出值得方法就是先通过reflect.ValueOf()获取参数值对象，然后再通过不同的具体方法获取到值对象，比如int和bool

示例3:

```
package main

import (
    "fmt"
    "reflect"
)

func main() {
    handleValue(1)
    handleValue(true)
}

func handleValue(data interface{}) {
    //获取类型对象
    d := reflect.ValueOf(data)
    //kind方法是获取类型
    fmt.Println(d.Kind())
    switch d.Kind() {
        case reflect.Invalid:
            //无效类型逻辑处理
            fmt.Println("无效类型")
        case reflect.Int, reflect.Int8, reflect.Int16, reflect.Int32, reflect.Int64:
            //取出值
            var myNum = d.Int()
            fmt.Println(myNum)
        case reflect.Bool:
            //取出bool值
            var myBool = d.Bool()
            fmt.Println(myBool)
    }
}
```

结构体中的属性和方法怎么获取呢？

获取结构体属性的个数是先ValueOf获取结构体值对象v后，用v.NumField()获取该结构体有几个属性，通过v.Field(i)来获取对应位置的属性的元类型。

示例代码4:

后续反射还有几个api和代码示例和具体应用场景，正在补。。。

25.变量作用域的区别

Go语言的变量作用域和Java中的一样，遵循最近原则，逐渐往外层找。

这个比较简单，就不做过多赘述了。

26.Go语言和Java语言字符串操作的区别

27.Go语言和Java语言IO操作的区别

28.Go语言中有匿名函数，有闭包，Java中没有(高阶函数用法)

函数也是一种类型，它可以作为一个参数进行传递，也可以作为一个返回值传递。

Go中可以定义一个匿名函数，并把这个函数赋值给一个变量

示例1：匿名函数赋值给变量

```
package main

import "fmt"

//定义一个匿名函数并赋值给myFun变量
var myFun = func(x,y int) int {
    return x+y
}

func main() {
    //调用myFun
    fmt.Println(myFun(1,2))
}
```

输出结果：

```
3
```

Go的函数内部是无法再声明一个有名字的函数的，Go的函数内部只能声明匿名函数。

示例2：

```
package main

import "fmt"

func main() {
    myFunc3()
}

func myFun1() {
    /*此处报错，函数内部不能声明带有名称的函数
    func myFunc2() {

    }
    */
}
```

```

}

func myFunc3() {
    //函数内部可以声明一个匿名函数，并把这个匿名函数赋值给f变量
    var f = func() {
        fmt.Println("Hi,boy!")
    }
    //调用f
    f()
    //如果不想赋值给变量，那就必须在最后加上(),表示立即执行
    func() {
        fmt.Println("Hello,girl!")
    }()//有参数可以写在这个小括号中
}

```

输出:

```

Hi,boy!
Hello,girl!

```

Go中有闭包的功能。(闭包是一个通用的编程概念，一些语言有，一些没有，javascript中就有这个概念，Java中没有)

闭包,通俗易懂的讲，就是你有一个A函数，A函数有一个a参数，然后在A函数内部再定义或者调用或者写一个B函数，这个B函数叫做闭包函数。B函数内部的代码可以访问它外部的A函数的a参数，正常A函数调用返回完毕，a参数就不能用了，可是闭包函数B函数仍然可以访问这个a参数，B函数能不受A函数的调用生命周期限制可以随时访问其中的a参数，这个能访问的状态叫做已经做了闭包，闭包闭的是把a参数封闭到了B函数中，不受A函数的限制。

也就是说，我们用程序实现一个闭包的功能，实质上就是写一个让外层的函数参数或者函数内变量封闭绑定到内层函数的功能。

接下来我们看代码示例：

```

package main

import "fmt"

//我们来看看实现闭包
func main() {
    var f = f1(100)
    f(100) //print 200
    f(100) //print 300
    f(100) //print 400
}

func f1(x int) func(int){
    //此时即使f1函数执行完毕，x也不会消失
    //x在func(y int)这个函数内一直存在并且叠加，
    //这里把x的值封闭到func(y int)这个返回函数中，使其函数一直能使用x的值
    //就叫做闭包，把x变量封闭到fun(y int)这个函数包内。
    return func(y int){
        x+=y
        fmt.Printf("x=%d\n",x)
    }
}

```

```
}
```

输出：

```
x=200
x=300
x=400
```

做下闭包的总结，如何实现一个闭包：

- 1.定义一个A函数，此函数返回一个匿名函数。（定义一个返回匿名函数的A函数）
- 2.把在A函数的b参数或A函数代码块中的b变量，放入匿名函数中，进行操作。
- 3.这样我们调用A函数返回一个函数，这个函数不断的调用就可以一直使用之前b参数，b变量，并且b值不会刷新，有点像在匿名函数外部自定义了一个b的成员变量（成员变量取自java中类的相关概念）

29.Go中的map和Java中的HashMap

Go中的map也是一个存储key-value，键值对的这么一种数据结构。

我们来看下如何使用：

如何创建一个map?(map是引用类型，默认值是nil，必须用make为其创建才能使用)

创建一个map必须要用make，否则会是nil

格式为: make(map[key类型]value类型)(下面有代码示例)

往Go中的map赋值添加元素用 【 map变量名称[key] = value 】 的方式

示例1：创建map以及添加元素

```
package main

import "fmt"

func main() {
    //创建一个map必须要用make，否则会是nil
    //格式为： make(map[key类型]value类型)
    //Java中： Map<String,Integer> myMap = new HashMap<>();
    myMap := make(map[string]int)
    //往Go中的map赋值添加元素用 【 map变量名称[key] = value 】 的方式
    //区别于Java中的： myMap.put("li_age",20);
    myMap["li_age"] = 20
    myMap["hong_age"] = 30
    //打印一下map
    fmt.Println(myMap)
}
```

我们从map中取值得格式为： 【 mapValue := map变量名[key]】

当我们填写的key在map中找不到时返回对应的value默认值，int是0，引用类型是nil

当我们的key取不到对应的值，而value的类型是一个int类型，我们如何判断这个0是实际值还是默认值呢

此时我们需要同时取两个值

通过map的key取出两个值，第二个参数为bool类型，false为该值不存在，true为成功取到值

参考下面：

示例2：从map中取值

```
package main

import "fmt"

func main() {
    //创建一个map必须要用make，否则会是nil
    //格式为： make(map[key类型]value类型)
    //Java中： Map<String,Integer> myMap = new HashMap<>();
    myMap := make(map[string]int)
    //往Go中的map赋值添加元素用 【 map变量名称[key] = value 】 的方式
    //区别于Java中的： myMap.put("li_age",20);
    myMap["li_age"] = 20
    myMap["hong_age"] = 30
    //打印一下map
    fmt.Println(myMap)
    //不存在的值
    fmt.Println(myMap["no"])

    //当我们的key取不到对应的值，而value的类型是一个int类型，我们如何判断这个0是实际值还是默认值呢
    //此时我们需要同时取两个值
    //通过map的key取出两个值，第二个参数为bool类型，false为该值不存在，true为成功取到值
    value,existsValue := myMap["no"]
    if !existsValue {
        fmt.Println("此值不存在")
    } else {
        fmt.Printf("value = %d",value)
    }
}
```

Go中因为返回值可以是两个，所以的map遍历很简单，不像java还得弄一个Iterator对象再逐个获取，它一次两个都能取出来，用for搭配range即可实现。

示例3：遍历

```
package main

import "fmt"

func main() {
    myMap := make(map[string]int)
    myMap["num1"] = 1
    myMap["num2"] = 2
    myMap["num3"] = 3
    myMap["num4"] = 4
    myMap["num5"] = 5
    myMap["num6"] = 6
    //遍历key,value
    for key,value := range myMap {
```



```

    fmt.Println(key,value)
}
//写一个参数的时候只取key
for key := range myMap {
    fmt.Println(key)
}
//如果只想取value，就需要用到之前的_标识符进行占位丢弃
for _,value := range myMap {
    fmt.Println(value)
}
}

```

删除函数：用内置函数delete删除

示例4：删除map元素

```

package main

import "fmt"

func main() {
    myMap := make(map[string]int)
    myMap["num1"] = 1
    myMap["num2"] = 2
    myMap["num3"] = 3
    myMap["num4"] = 4
    myMap["num5"] = 5
    myMap["num6"] = 6
    //第二个参数为删除的key
    delete(myMap, "num6")
    //此时已经没有值了，默认值为0
    fmt.Println(myMap["num6"])
}

```

在Java中有一些复杂的Map类型，比如：

```
Map<String,Map<String,Object>> data = new HashMap<>();
```

实际上，在Go语言中，也有复杂的类型，我们举几个代码示例

示例5：

```

package main

import "fmt"

func main() {
    //由map组成的切片
    //第一部分 make[] 声明切片
    //第二部分 map[string]int 声明该切片内部装的单个类型是map
    //第三部分 参数 5 表示该切片的长度和容量都是5
    //长度是用索引能取到第几个元素，索引不能超过长度-1，分配长度后都是默认值，int是0，引用类型是nil
    //容量至少比长度大，能索引到几个+未来可添加元素个数(目前没有任何东西，看不见)= 切片容量
}

```

```

//make([]切片类型,切片长度,切片容量)
//make([]切片类型,切片长度和容量等同)
slice := make([]map[string]int,5,10)
slice0 := make([]map[string]int,0,10)
//我们看看打印的东西
fmt.Println("slice=",slice)
fmt.Println("slice=0",slice0)

/* 先看这段
//因为有5个长度,所以初始化了5个map,但是map没有通过make申请内容空间,所以报错nil map
//slice[0]["age"] = 10;//报错
//下面不报错
slice[0] = make(map[string]int,10)
slice[0]["age"] = 19
fmt.Println(slice[0]["age"])
*/
}

```

输出结果:

```

slice= [map[] map[] map[] map[] map[]]
slice=0 []
19

```

接下来继续看代码:

```

package main

import "fmt"

func main() {
    //由map组成的切片
    //第一部分 make[] 声明切片
    //第二部分 map[string]int 声明该切片内部装的单个类型是map
    //第三部分 参数 5 表示该切片的长度和容量都是5
    //长度是用索引能取到第几个元素,索引不能超过长度-1,分配长度后都是默认值,int是0,引用类型是
    nil
    //append元素到切片时,是添加到最末尾的位置,当元素未超过容量时,都是用的同一个底层数组
    //超过容量时会返回一个新的数组
    //make([]切片类型,切片长度,切片容量)
    //make([]切片类型,切片长度和容量等同)
    slice := make([]map[string]int,5,10)
    slice0 := make([]map[string]int,0,10)
    //我们看看打印的东西
    fmt.Println("slice=",slice)
    fmt.Println("slice=0",slice0)

    /* 先看这段
    //因为有5个长度,所以初始化了5个map,但是map没有通过make申请内容空间,所以报错nil map
    //slice[0]["age"] = 10;//报错
    //下面不报错
    slice[0] = make(map[string]int,10)
    slice[0]["age"] = 19
    fmt.Println(slice[0]["age"])
    */
}

```

```
*/  
  
}
```

输出:

```
panic: assignment to entry in nil map
```

看下面这个报错:

```
package main  
  
import "fmt"  
  
func main() {  
    //由map组成的切片  
    //第一部分 make[] 声明切片  
    //第二部分 map[string]int 声明该切片内部装的单个类型是map  
    //第三部分 参数 5 表示该切片的长度和容量都是5  
    //长度是用索引能取到第几个元素，索引不能超过长度-1，分配长度后都是默认值，int是0，引用类型是  
    nil  
    //append元素到切片时，是添加到最末尾的位置，当元素未超过容量时，都是用的同一个底层数组  
    //超过容量时会返回一个新的数组  
    //make([]切片类型,切片长度,切片容量)  
    //make([]切片类型,切片长度和容量等同)  
    slice := make([]map[string]int,5,10)  
    slice0 := make([]map[string]int,0,10)  
    //我们看看打印的东西  
    fmt.Println("slice=",slice)  
    fmt.Println("slice0=",slice0)  
  
    /* 先看这段  
    //因为有5个长度，所以初始化了5个map，但是map没有通过make申请内容空间，所以报错nil map  
    //slice[0]["age"] = 10;//报错  
    //下面不报错  
    slice[0] = make(map[string]int,10)  
    slice[0]["age"] = 19  
    fmt.Println(slice[0]["age"])  
    */  
    ///*  
    //因为初始化了0个长度，所以索引取不到值，报index out of range  
    slice0[0]["age"] = 10;  
  
    //*/  
}
```

输出:

```
slice= [mappanic: runtime error: index out of range
```

接下来我们看一个: 类似于Java中常用的map类型

```
package main
```

```
import "fmt"

func main() {
    //类似于Java中的Map<String,HashMap<String,Object>>
    var myMap = make(map[string]map[string]interface{},10)
    fmt.Println(myMap)
    //记得make
    myMap["li_ming_id_123"] = make(map[string]interface{},5)
    myMap["li_ming_id_123"]["school"] = "清华大学"

    fmt.Println(myMap)
}
```

输出:

```
map[]
map[li_ming_id_123:map[school:清华大学]]
```

30.Go中的time时间包模块和Java中的时间API使用区别

Go中关于时间处理的操作在time包中

1.基本获取时间信息

参考如下代码示例:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    //获取当前时间
    now := time.Now()
    //获取当前年份
    year := now.Year()
    //获取当前月份
    month := now.Month()
    //获取当前 日期
    day := now.Day()
    //获取当前小时
    hour := now.Hour()
    //获取当前分钟
    min := now.Minute()
    //获取当前秒
    second := now.Second()

    //获取当前时间戳, 和其它编程语言一样, 自1970年算起
    timestamp := now.Unix()
    //纳秒时间戳
    ntimestamp := now.UnixNano()
}
```

```

    fmt.Println("year=",year)
    fmt.Println("month=",month)
    fmt.Println("day=",day)
    fmt.Println("hour=",hour)
    fmt.Println("min=",min)
    fmt.Println("second=",second)
    fmt.Println("timestamp=",timestamp)
    fmt.Println("ntimestamp=",ntimestamp)
}

```

2.格式化时间

Go的时间格式化和其它语言不太一样，它比较特殊，取了go的出生日期作为参数标准

参考如下代码示例：

```

package main

import (
    "fmt"
    "time"
)

func main() {
    //获取当前时间
    now := time.Now()

    //2006-01-02 15:04:05这个数值是一个标准写死的，只要改格式符号即可
    fmt.Println(now.Format("2006-01-02 15:04:05"))
    fmt.Println(now.Format("2006/01/02 15:04:05"))
    fmt.Println(now.Format("2006/01/02"))//年月日
    fmt.Println(now.Format("15:04:05"))//时分秒
}

```

31.Go和Java关于Socket编程的对比

32.聊聊Go语言如何连接Mysql数据库

33.聊聊Go语言如何使用Redis

34.Go中的依赖管理--Module,对比Java的maven

35.Go的协程高并发支持与Java的区别

36.Go的性能调优和Java的性能调优

37.Go的测试API与Java的单元测试

38.自定义类型Type

39.Go的参数值传递与引用传递

接下来我们讲一下Go中的参数传递原理。

关于参数传递是一个什么概念呢，参数传递相关的知识是在研究当调用一个函数时，把外部的一个变量传入函数内，在函数内修改这个参数是否会对外部的参数变量的值有影响。参数传递用在的一个地方是函数的参数传递。（还有方法的接收器参数传递）

比如李明今天没有写作业，到了学校后匆匆忙忙的找小红要作业本(小红的作业本为方法调用处传入的参数)，想要抄一抄补上，所以李明有一个抄作业的任务(抄作业的任务为函数)，那么他有两个选择可以完成抄作业的任务。

第一个是直接拿过来小红的作业本开始抄，这在函数中叫做引用传递，因为如果小明抄的时候不小心桌子上的水打翻了，弄湿了小红的作业本，小红的作业本就真湿了，没法交了。

第二个是用打印机把小红的作业打印一份，然后拿着打印的那份抄，这叫做值传递，也就是说我拷贝一份值来用，那么我在抄作业(任务函数内)无论怎么弄湿小红的作业本，小红真正的自己的作业本也不受到影响。

在编程语言的函数中，如果是值传递，则是一个拷贝，在方法内部修改该参数值无法对其本身造成影响，如果是引用传递的概念，则可以改变其对象本身的值。

在Go语言中只有值传递，也是是说，无论如何Go的参数传递的都是一个拷贝。

重点来了：

Go中的值传递有两种类型，如下：

1.第一种值传递是具体的类型对象值传递，可能是int,string,struct之类的。

在此时，如果我们要自定义一个**struct**类型，传入参数中，可能遇到一个坑，因为是值传递，所以会拷贝一个**struct**对象，如果这个对象占内存比较大，而且这个函数调用频繁，会大量的拷贝消耗性能资源。

2.第二种传递是叫指针参数类型的值传递，此时参数是一个指针类型，到具体的方法中，我们的参数也要用指针类型的参数接受，但是此时Go语言的内部做了一个黑箱操作。

举例(下面还有完整可执行代码示例，先文字和伪代码举例)：

我们有一个类型为Boy的结构体,还有一个方法Mod

```
func Mod(b *Boy){  
  
}
```

这个Mod方法的参数是一个指针类型的Boy对象，

我们要调用的时候应该这样传参数：

```
var boy = Boy{}  
//用&取boy对象的指针地址，然后传入Mod方法  
Mod(&boy)
```

我们看看下面的代码示例：

```

package main

import "fmt"

// Boy 结构体
type Boy struct {
    name string
    age  int
}

func Mod(b *Boy) {
    //这个是获取调用方法传入的参数的地址值
    fmt.Printf("b的值(之前boy的地址)是%p\n", b)
    //这个是获取本函数中 b这个指针变量的地址
    fmt.Printf("b这个指针自己的地址是%p\n", &b)
    //打印值
    //这里自动转换使指针可以直接点出来属性
    fmt.Println(b.name, b.age)
}

func main() {
    boy := Boy{"li_ming", 20}
    fmt.Printf("main函数中的boy地址是%p\n", &boy)
    //将boy的地址 放入Mod函数的参数中
    Mod(&boy)
    //注意!!! 下面有黑箱操作:
    /* //在&boy并放入Mod传递的过程中实际上做了如下黑箱操作
    b := new(Boy)    //创建一个名为b的类型为Boy的指针变量
    b = &boy         //把boy的地址存入b这个指针变量内
    //接着把b放入func Mod(b *Boy)的参数中, 然后, 开始执行Mod方法。
    fmt.Println(b.name, b.age)
    fmt.Printf("b的地址是%p\n", &b)
    fmt.Printf("b的值是%p\n", b)

    //输出结果
    //main函数中的boy地址是:0x10aec0c0
    //li_ming 20
    //b的地址是:0x10ae40f8
    //b的值是:0x10aec0c0
    */

    /*//以下代码无用, 是指为了加深理解new, 可以试试输出结果
    boy2 := new(Boy)
    fmt.Printf("main函数中新new的boy2地址是%p\n", boy2)
    boy2.name = "xiaohong"
    boy2.age = 18
    Mod(boy2)
    */
}

```

所以, Go中的参数传递所有的都是值传递,

只不过值传递中, 值可以是指针类型, 是创建了一个新的指针存储原来参数(这个参数是原对象的地址)的值。

所以你用原对象的地址改它的属性，是有点类似于引用类型传递的效果的。

为啥说指针类型也是值传递，因为他还是创建了一个新的指针对象，值传递就是拷贝，拷贝就得创建对象，只不过这个新的指针变量存储的值是原来的参数对象的地址。

最后总结一下：

1.Go的参数传递都是值传递。

2.指针类型的值传递可以改变原来对象的值。

3.make和new从底层原理上创建的所有对象都是指针对象，所以make和new创建出来的slice,map,chan或者其它任何对象都是指针传递，改变值后都可以使原来的对象属性发生变化。

40.结构体转JSON

41.Go如何搭建HTTP-Server

42.Go如何搭建HTTP-Client

43.Go如何设置使用的CPU个数

Go语言天生支持高并发，其中一个体现就是如果你的Go程序不设置并发时使用的最大cpu核数的话，在高并发情况下Go会自动把所有CPU都用上，跑满。

拓展阅读：

我们简单理解一下cpu（懂得可以跳过）

举例：比如有一个专门做财务的公司(计算机)，他们的赚钱业务很简单(计算机工作)，就是帮别人做算术题(计算机工作的具体任务)，加减乘除之类的算术题，现在公司有4个员工(物理意义上的4个cpu核数)，有4本算数书(4个进程)，每本书有10道题(线程)，一共有40道算术题要算(40个线程任务)，于是4个人一起干活，在同一时间，有4道算术题被计算，最后大致上每个人算了10道算数题。

第二天，有8本算术书(8个进程)，他们为了快速完成任务，规定一人(每个人是一个物理cpu核数)管2本算数书(单物理cpu内部实际上是管理的两个不同的算数书，也就是相当于有两个不同的逻辑cpu)，为的就是如果第一本做烦了可以换着做第2本，混合着做，最后都做完就可以。

cpu是进行最终二进制计算的核心计算器，cpu核数是有两个概念，一个是真实世界的物理硬件核数，比如4核cpu，就是有4个物理硬件内核，然而我们在生产环境的linux服务器上top的时候，出现的cpu个数实际上是逻辑cpu数，有可能linux服务器只有4核物理cpu，可是每个物理cpu分为两个逻辑cpu，这个时候我们在linux上top看的时候就是有8个cpu信息行数据。

我们回顾一下Java，Java运行时我们一般管理的都是线程数，而所有的Java线程均在JVM这个虚拟机进程中，于是在高并发情况下，当cpu资源充足时，我们需要根据cpu的逻辑核数来确定我们的线程池线程数(在高并发环境下一定要设置优化线程数啊!!!线程池就能设置线程数!!!)，比如我们是4个物理cpu,每个双核逻辑，一共逻辑八核cpu,此时，比如我们要做并发定时任务，这台服务器没有其它程序，8个cpu全都给我们自己用，那么我们的线程数最少也要设置成8，再细化，我们得根据程序执行的任务分别在cpu计算(正常处理程序业务逻辑)的耗时和cpuIO耗时（IO耗时比如查mysql数据库数据），假如我们定时跑批任务一个任务计算用时0.2秒，查数据库0.8秒(自己可以写程序监测)，那么可以参考如下公式：

总任务耗时/cpu耗时 = 多少个线程(每个逻辑cpu)

我们算出每个逻辑cpu要跑多少个线程后再乘以逻辑cpu的个数，就能算出来了

如下：

$(0.2+0.8)/0.2=5$ 个线程(每个逻辑cpu)

5*8 = 40

于是我们在线程池的时候应该这么写：

```
ExecutorService fixedThreadPool = Executors.newFixedThreadPool(40);
```

至于公式为什么要这么写，是因为IO操作的时候，cpu是空闲的，也就是说，0.8秒数据库操作的时候，cpu都是空闲的，那么我们就多开几个线程让cpu在这0.8秒的时候工作，开几个呢，要等0.8秒，一个任务cpu要计算0.2秒， $0.8/0.2=4$ (个)，可是这个逻辑cpu还有一个主线程在那0.8秒上等着结果呢，所以是 $4+1=5$ （个）线程。

上述我们回顾了Java中的线程数和CPU核数相关，接下来我们来看Go语言。

我们下面来仔细讲讲Go中的goroutine(实际是协程)，是如何天然的支持高并发的，它与Java中的线程Thread又有什么区别，为什么它比线程能更好的支持高并发。

44.初始化结构体，匿名结构体，结构体指针(再讲)

45.方法中的值接收和指针接收的区别(方法进阶细节讲解)

我们之前讲了如果给一个类型绑定上一个接受者，就可以为这个类型添加一个这个类型独有的函数，只有这个类型对象自己能调用的函数，这个特殊的函数叫方法。

现在，我们讲一下方法关于传递接受者（自身引用）的进阶玩法。

Go语言中的参数传递

46.基于包模块的Init函数

47.Go 语言中的初始化依赖项

48.slice相关知识

slice的中文意思是切片。

要想理解切片，我们首先要理解数组。

数组是一个长度不能变化的容器，存储同一数据类型的数据。

比如：int数组

```
[1,2,3,4,5,6]
```

切片是对数组中一截，一小段，一个子集的地址的截取，切片存储的是它指向的底层数组中的一小截数据的地址，切片中不存数据，创建切片也不会把数组中的数据copy一份，切片只是存储着数组中一部分连续的数据的地址，切片的每一个元素实际上都指向具体的数组的中一个元素。

切片内部包含三个元素：

1.底层数组（它指向的是哪一个数组）

我们要理解底层数组是什么，先举例：

[1,2,3]这是一个int数组，其中元素1的地址是 0x0001,元素2的地址是0x0002，元素3的地址是0x0003。

那么如果我们创建一个通过数组[1,2,3]创建一个切片x。

这个x里面存储的并不是拷贝的另外一份新的[1,2,3]。

切片x实际上是这样子的：

[0x0001,0x0002,0x0003]

当我们取出x[0]的时候，它操作的实际上是0x0001这个地址的元素，而这个地址实际上就是数组[1,2,3]中的1的地址。

也就是说，当我们修改了数组[1,2,3]中的1后，比如 0x0001 = 5,切片x中的0元素的取值自动也不一样了，因为0x0001地址上存储的值已经被改成5了，而x[0]实际上还是0x0001,所以此时取出x[0],得到的就是5。

切片存储的每一个元素实际上是它指向的底层数组的每一个元素的地址。

也就是说切片是一个引用类型，它不存储元素，不拷贝元素，它存储数组元素的引用，通过修改切片会修改原来数组的值。

2.切片的长度

这个切片中有有几个元素，指向了数组中的几个连续的元素。

3.切片的容量

从切片在底层数组的起始下标(切片的首个元素)到底层数组的最后一个元素，一共有几个元素，切片的容量就是几。

例如：(下面先用伪代码示例，后面有具体可执行代码)

原数组：a = [1,2,3,4,5,6,7,8]

切片：b = a[2:5] 从数组a的下标为2的开始，也就是具体数值是3开始，截取到下标为5，下标为5的是6，因为切片截取是左开右闭，所以切片中包括下标为2的数值3，不包含下标为5的数值6。

切片存储的地址指向的数据是：[3,4,5]

因为3，4，5有三个数，所以切片的长度是3。

因为从切片的起始元素3到底层数组的末尾元素8之间有6个元素，所以切片的容量是6。

修改切片实际上是修改切片指向的底层数组中的值。

49.Go中类似于函数指针的功能

Go中要实现函数指针非常简单。

因为Go中的函数也是一种类型。

所以我们只要声明一个变量，把某一个函数赋值给这个变量，就能实现函数指针的效果。

如下代码示例：

```
package main

import "fmt"

//加法
func myAddFun(x,y int) int {
    return x+y
}

//减法
```

```
func mySubFun(x,y int) int{
    return x-y;
}
//函数变量(类似于函数指针)
var myPointFun func(x,y int) int

func main() {
    //加法函数赋值给该函数变量，相当于函数指针指向加法函数
    myPointFun = myAddFun
    fmt.Printf("a+b = %d\n",myPointFun(10,20))
    //减法函数赋值给该函数变量，相当于函数指针指向减法函数
    myPointFun = mySubFun
    fmt.Printf("a-b = %d\n",myPointFun(100,50))
}
```

输出：

```
a+b = 30
a-b = 50
```

50.Go有没有注解

原生的Go语言的SDK是不支持注解功能的，但是有一些其它的第三方机构为了实现自己的某些功能需求编写了一些自定义的注解。

51.Go不能做大数据相关的开发

因为大数据的一些底层都是Java开发的，用Java实现接口开发功能非常方便快捷，对于Go语言的支持比较少，另外就是一些算法库像numpy,pandas和一些机器学习，深度学习算法库Python支持的比较好，对于Go的支持很不好。

52.Go没有泛型

Go中不支持泛型(明确)

53.Go如何产生随机数(随机数和种子)

54.Go如何打类似于(java jar那种依赖包).a的工具依赖包(有了Module后不用这个了)

Go中也有很多通过命令来完成辅助开发的工具，就像Java中jdk中的java javac javap等指令那种命令工具。

比如有go build xxx命令，go clean xxx命令，go run xxx命令.....

Java中打jar包可以通过IDEA集成开发环境图形界面化直接打包，也可以使用jar命令在命令行操作中(使用不同的参数)进行打包。

与java jar命令打包对应的Go的命令是 go install,这个go install 也类似于 maven 中的install,它会把打成的.a后缀名结尾的工具包文件

放入\${GOPATH}/pkg下。

具体使用如下示例：

注意：使用go install之前必须在操作系统的环境变量中定义\${GOPATH}这个环境变量

- 1.查看我们当前的操作系统中环境变量有没有定义GOPATH。
- 2.查看\${GOPATH}目录下是否有src, pkg, bin目录，并且保证我们的代码是在src下的。
- 3.打开一个命令行窗口，比如windows是cmd打开一个dos窗口。
- 4.我们在最开始之前已经把go的安装包下的包含Go操作指令的bin目录配置在了PATH环境变量中，所以此时我们可以不用管目录直接使用go install 命令。

5. 例如目录结构如下：

```
com.mashibing.gotest
-----mygopackge
                        MyUtil.go
```

记住一点，此时MyUtil中不能是main包，也不能有main函数，否则打不出来.a结尾的依赖包。

此时编写执行命令：

```
go install com/mashibing/gotest/mygopackge
```

此指令运行时，首先会去找\${GOPATH}目录

然后把后面的com/mashibing/gotest/mygopackge拼接上去

也就是\${GOPATH}/com/mashibing/gotest/mygopackge

然后会把\${GOPATH}/com/mashibing/gotest/mygopackge下的所有.go文件，比如MyUtil.go全部打包压缩进mygopackge.a文件

最后会把mygopackge.a放入\${GOPATH}/pkg/\${标示操作系统的一个名字(这个不重要)}/com/mashibing/gotest/下。

最终.a文件存储的结构是这样的：

```
${GOPATH}/pkg/com/mashibing/gotest/mygopackge.a
```

55.Go中的依赖管理Module使用

1.什么是GoModule?(Go中Module和包的区别?)

首先我们要理解一下Go的Module是一个什么概念？

我先简单的说一下，Go中的Module是GoSDK1.11后提出的一个新的类似于包的机制，叫做模块，在1.13版本后成熟使用，GoSDK中Module功能是和相当于一个包的增强版，一个模块类型的项目在根目录下必须有一个go.mod文件，这个模块项目内部可以有很多个不同的包，每个包下有不同的代码，我们下载依赖的时候是把这个模块下载下来(模块以压缩包(比如zip)的形式存储在\${GOPATH}/pkg/mod/cache/下，源码文件也会在\${GOPATH}/pkg/mod/下)。

我们导入模块的时候只需要引入一次，使用模块中不同的包的时候可以通过import模块下不同的包名，来引入不同包的功能。

比如下面的结构

```
-----com.mashibing.module
-----package1
```

```
-----test1.go
-----package2
-----test2.go
```

然后我们只需要在go.mod中引入这一个模块，就能在import的时候任意引入package1或package2。

2.为什么要使用GoModule?

1).团队协作开发中对于包的版本号的管理

在没有Module之前，我们都是把自己写的Go程序打成包，然后别的程序引用的话引入这个包。

可是在开发中这些包的版本有个明显的不能管理的问题。

比如我怎么知道这个包是今天开发的最新版还是明天开发的，我在团队协同开发中怎么把别人写的最新版本的包更新到我的项目中。

2) 便于开发中的依赖包管理

其次，我们在开发中下载了别人的项目，怎么快速的观察有哪些依赖包，如何快速的把所有依赖包从仓库中下载下来，都是一个问题，

这两个问题就可以通过观察项目根目录下的go.mod文件的依赖模块列表和执行go mod download命令快速从第三方模块仓库中下载依赖包来完成。

3).隔离管理不同类别的项目

有了Module后，我们可以把我们自己的项目和系统的项目隔离管理，我们的项目不用必须放在\${GOPATH}/src下了

3.哪些项目能使用GoModule?

一个GoModule项目要想引入其它依赖模块，需要在根目录下的go.mod中添加对应的依赖模块地址。

注意：！！！重点来了！！！！

GoModule只能引用同样是Module类型的项目，经常用于引用内部自己的项目。

像maven仓库一样引用开源模块的依赖也是一个特别常用的场景。

不过我们需要修改代理地址访问国内的第三方GoModule提供商。

<https://goproxy.cn/>是一个国内的可访问的GoModule依赖仓库，类似于Java中maven中央仓库的概念。

4.GoModule的版本问题?

我们使用Go module之前，首先要保证我们的Go SDK是在1.13以及以上版本。(Go1.11以上就可以使用Module,但是需要设置一些开启等，1.13后默认开启)

因为在1.13版本上官方正式推荐使用，成为稳定版了。

Go也有代码仓库，比如可以使用github作为go项目的代码仓库，Go语言本身就提供了一个指令 go get 来从指定的仓库中

拉取依赖包和代码，不过go get这个指令在开启模块功能和关闭模块功能下用法不一样，下面有开启模块下的用法。

5.GoModule和Java中Maven的区别？

Go中的Module和Java中的Maven不同：

首先，Module是官方的SDK包自带的，它并非像maven一样还得安装maven插件之类的。

关于中央依赖仓库，Go和Java中的概念是类似的，都是国内的第三方提供的。

6.如何开启GoModule?(GO111MODULE)

具体我们如何使用Module呢？

我们首先要检查我们的GoSDK版本是1.11还是1.13之上。

如果是1.11的话我们需要设置一个操作系统的中的环境变量，用于开启Module功能，这个是否开启的环境变量名是GO111MODULE，

他有三种状态：

第一个是on 开启状态，在此状态开启下项目不会再去\${GOPATH}下寻找依赖包。

第二个是off 不开启Module功能，在此状态下项目会去\${GOPATH}下寻找依赖包。

第三个是auto自动检测状态，在此状态下会判断项目是否在\${GOPATH}/src外，如果在外边，会判断项目根目录下是否有go.mod文件，如果均有则开启Module功能，如果缺任何一个则会从\${GOPATH}下寻找依赖包。

GoSDK1.13版本后GO111MODULE的默认值是auto，所以1.13版本后不用修改该变量。

注意：在使用模块的时候，GOPATH 是无意义的，不过它还是会把下载的依赖储存在 \${GOPATH}/src/mod 中，也会把 go install 的结果放在 \${GOPATH}/bin 中。

windows

```
set GO111MODULE=on
```

linux

```
export GO111MODULE=on
```

7.GoModule的真实使用场景1：

接下来我们代入具体的使用场景：

今天，小明要接手一个新的Go项目，他通过GoLand中的git工具，从公司的git仓库中下载了一个Go的项目。(下载到他电脑的非\${GOPATH}/src目录，比如下载到他电脑的任意一个自己的工作空间)

此时他要做的是：

1).先打开项目根目录下的go.mod文件看看里面依赖了什么工具包。(这个就是随便了解一下项目)

2).Go的中央模块仓库是Go的官网提供的，在国外是<https://proxy.golang.org>这个地址，可是在国内无法访问。

我们在国内需要使用如下的中央模块仓库地址：<https://goproxy.cn>

我们Go中的SDK默认是去找国外的中央模块仓库的，如何修改成国内的呢？

我们知道，所有的下载拉取行为脚本实际上是从 go download 这个脚本代码中实现的，而在这个脚本中的源码实现里，肯定有一个代码是写的是取出操作系统中的一个环境变量，这个环境变量存储着一个地址，这个地址代表了去哪个中央模块仓库拉取。

在GoSDK中的默认实现里，这个操作系统的环境变量叫做GOPROXY，在脚本中为其赋予了一个默认值，就是国外的proxy.golang.org这个值。

我们要想修改，只需要在当前电脑修改该环境变量的值即可：

(注意，这个变量值不带https,这只是一个变量，程序会自动拼接https)

windows

```
set GOPROXY=goproxy.cn
```

linux

```
export GOPROXY=goproxy.cn
```

3).切换到项目的根目录，也就是有go.mod的那层目录，打开命令行窗口。

执行 download指令(下载模块项目到\${GOPATH}/pkg/mod下)

```
go mod download
```

4).如果不报错，代表已经下载好了，可以使用了，此时在项目根目录会生成一个go.sum文件。

一会再讲解sum文件。

5).此时可以进行开发了。

8.GoModule的真实使用场景2：

场景2：我们如何用命令创建一个Module的项目，(开发工具也能手动创建)。

切换到项目根目录，执行如下指令：

```
go mod init 模块名(模块名可不写)
```

然后会在根目录下生成一个go.mod文件

我们看看这个go.mod文件长啥样？

```
// 刚才init指令后的模块名参数被写在module后了
module 模块名
//表示使用GoSDK的哪个版本
go 1.14
```

修改go.mod文件中的依赖即可。

我们有两种方式下载和更新依赖：

1.修改go.mod文件，然后执行go mod down 把模块依赖下载到自己\${GOPATH}/pkg/mod下，这里面装的是所有下载的module缓存依赖文件，其中有zip的包，也有源码，在一个项目文件夹下的不同文件夹下放，还有版本号文件夹区分，每个版本都是一个文件夹。

2.直接在命令行使用go get package@version 更新或者下载依赖模块，升级或者降级模块的版本。(这里是开启模块后的go get指令用法)

例如：

```
go get github.com/gin-contrib/sessions@v0.0.1
```

这个指令执行过后，会自动修改go.mod中的文件内容，不需要我们手动修改go.mod文件中的内容。

9.go.mod文件详解

接下来我们讲讲核心配置文件go.mod

go.mod内容如下：

```
//表示本项目的module模块名称是什么,别的模块依赖此模块的时候写这个名字
module test
//表示使用GoSDK的哪个版本
go 1.14
//require中声明的是需要依赖的包和包版本号
require (
    //格式如下: 需要import导入的模块名 版本号
    //          需要import导入的模块名2 版本号2
    //          ...
    github.com/gin-contrib/sessions v0.0.1
    github.com/gin-contrib/sse v0.1.0 // indirect
    github.com/gin-gonic/gin v1.4.0
    github.com/go-redis/redis v6.15.6+incompatible
    github.com/go-sql-driver/mysql v1.4.1
    github.com/golang/protobuf v1.3.2 // indirect
    github.com/jinzhu/gorm v1.9.11
    github.com/json-iterator/go v1.1.7 // indirect
    github.com/kr/pretty v0.1.0 // indirect
    github.com/mattn/go-isatty v0.0.10 // indirect
    github.com/sirupsen/logrus v1.2.0
    github.com/ugorji/go v1.1.7 // indirect
)
//replace写法如下,表示如果项目中有引入前面的依赖模块,改为引用=>后面的依赖模块,
//可以用于golang的国外地址访问改为指向国内的github地址,当然你在上面require直接写github就不用在这里replace了
replace (
    golang.org/x/crypto v0.0.0-20190313024323-a1f597ede03a =>
    github.com/golang/crypto v0.0.0-20190313024323-a1f597ede03a
)
//忽略依赖模块,表示在该项目中无论如何都使用不了该依赖模块,可以用于限制使用某个有bug版本的模块
exclude(
    github.com/ugorji/go v1.1.7
)
```

注: go.mod 提供了module、require、replace和exclude四个命令

module语句指定包的名字(路径)

require语句指定的依赖项模块

replace语句可以替换依赖项模块

exclude语句可以忽略依赖项模块

上面github.com/ugorji/go v1.1.7 // indirect 有 indirect和非indirect

indirect代表此模块是间接引用的，中间隔了几个项目

这个不用特殊写，可以注释写便于识别和开发

10.GoModule有哪些命令？如何使用？

Go有如下关于Module的命令：

```
//go mod 命令：
download //下载依赖模块到${GOPATH}/pkg/mod
edit      //一系列参数指令用于操作go.mod文件，参数太多，具体下面有例子
graph     //输出显示每一个模块依赖了哪些模块
init      //在一个非module项目的根目录下创建一个go.mod文件使其变为一个module管理的项目
tidy      //根据项目实际使用的依赖包修改(删除和添加)go.mod中的文本内容
vendor    //在项目根目录创建一个vender文件夹 然后把${GOPATH}/pkg/mod下载的本项目需要的依赖
          模块拷贝到本项目的vender目录下
verify    //校验${GOPATH}/pkg/mod中的依赖模块下载到本地后是否被修改或者篡改过
why       //一个说明文档的功能，用于说明一些包之间的为什么要这么依赖。(没啥用)
```

0). init和download

我们之前在案例中讲了init,download指令，这里不再赘述

1).go mod edit

是指在命令行用指令通过不同的参数修改go.mod文件，这个指令必须得写参数才能正确执行，不能空执行go mod edit

参数1：-fmt

```
go mod edit -fmt
```

格式化go.mod文件，只是格式规范一下，不做其它任何内容上的修改。

其它任何edit指令执行完毕后都会自动执行-fmt格式化操作。

这个使用场景就是我们如果不想做任何操作，就想试试edit指令，就只需要跟上-fmt就行，因为单独不加任何参数

只有go mod edit后面不跟参数是无法执行的。

我们如何升级降级依赖模块的版本，或者说添加新的依赖和移除旧的依赖呢

参数2：-require=path@version / -droprequire=path flags

添加一个依赖

```
go mod edit -require=github.com/gin-contrib/sessions@v0.0.1
```

删除一个依赖

```
go mod edit -droprequire=github.com/gin-contrib/sessions@v0.0.1
```

这两个和go get package@version 功能差不多，但是官方文档更推荐使用go get来完成添加和修改依赖（go get 后的package和上面的path一个含义，都是模块全路径名）

参数3: -exclude=path@version and -dropexclude=path@version

排除某个版本某个模块的使用, 必须有该模块才可以写这个进行排除。

```
go mod edit -exclude=github.com/gin-contrib/sessions@v0.0.1
```

删除排除

```
go mod edit -dropexclude=github.com/gin-contrib/sessions@v0.0.1
```

简单来说, 执行这两个是为了我们在开发中避免使用到不应该使用的包

.....还有好几个, 基本很少用, 省略了

2).go mod graph

命令用法: 输出每一个模块依赖了哪些模块 无参数, 直接使用, 在项目根目录下命令行执行

```
go mod graph
```

比如:

模块1 依赖了模块a

模块1 依赖了模块b

模块1 依赖了模块c

模块2 依赖了模块x

模块2 依赖了模块z

如下是具体例子:

```
C:\${GOPATH}\file\project>go mod graph
file\project github.com/edgexfoundry/go-mod-bootstrap@v0.0.35
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35
github.com/BurntSushi/toml@v0.3.1
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35 github.com/edgexfoundry/go-mod-configuration@v0.0.3
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35 github.com/edgexfoundry/go-mod-core-contracts@v0.1.34
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35 github.com/edgexfoundry/go-mod-registry@v0.1.17
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35 github.com/edgexfoundry/go-mod-secrets@v0.0.17
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35 github.com/gorilla/mux@v1.7.1
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35 github.com/pelletier/go-toml@v1.2.0
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35
github.com/stretchr/testify@v1.5.1
github.com/edgexfoundry/go-mod-bootstrap@v0.0.35 gopkg.in/yaml.v2@v2.2.8
github.com/edgexfoundry/go-mod-configuration@v0.0.3
github.com/cenkalti/backoff@v2.2.1+incompatible
github.com/edgexfoundry/go-mod-configuration@v0.0.3
github.com/hashicorp/consul/api@v1.1.0
```

3).go mod tidy

根据实际项目使用到的依赖模块，在go.mod中添加或者删除文本引用

有一个参数可选项 -v 输出在go.mod文件中删除的引用模块信息

比如我们项目用到一个模块，go.mod中没写，执行后go.mod中就会添加上该模块的文本引用。

如果我们在go.mod中引用了一个模块，检测在真实项目中并没有使用，则会在go.mod中删除该文本引用。

使用如下：

```
go mod tidy -v
```

输出：

```
unused github.com/edgexfoundry/go-mod-bootstrap
```

输出表示检测项目没有使用到该模块，然后从go.mod中把该包的引用文字给删除了。

4).go mod vander

该指令会在项目中建立一个vender目录，然后把\${GOPATH}/pkg/mod中下载的依赖拷贝到项目的vender目录中，方便管理和方便在idea中引用依赖。 -v参数可以在控制台输出相关的结果信息

```
go mod vander -v
```

5).go mod verify

验证下载到\${GOPATH}/pkg/mod中的依赖模块有没有被修改或者篡改。

结果会输出是否被修改过

```
go mod verify
```

比如输出：

```
all modules verified
```

这个是所有模块已经验证，代表没有被修改，如果被修改，会提示哪些被修改。

6).go mod why

这个没啥用，说白了就是一个解释文档，输入参数和依赖他说明哪些包为啥要依赖这些包，不用看它，用处不大。

11.go.sum详细讲解

1).go.sum什么时候会更新或者新建生成？

当我们通过go mod download 下载完依赖模块或者go get package@version更新了依赖包的时候，会检查根目录下有没有一个叫go.sum的文件，没有的话则创建一个并写入内容，有的话会更新go.sum中的内容。

2).go.sum是用来做什么的?

go.sum的作用是用来校验你下载的依赖模块是否是官方仓库提供的, 对应的正确的版本的, 并且中途没有被黑客篡改的。

go.sum主要是起安全作用和保证依赖的版本肯定是官方的提供的那个版本, 版本确认具体是确认你下载的那个模块版本里面的代码的和官方提供的模块的那个版本的代码完全相同, 一字不差。

通过go.sum保证安全性是很有必要的, 因为如果你的电脑被黑客攻击了, 黑客可以截取你对外发送的文件, 也可以修改发送给你的文件, 那么就会产生一个问题:

本来的路径应该是这样的: 第三方模块依赖库----->你的电脑

结果中间有黑客会变成这样:

第三方模块依赖库----->黑客修改了依赖库中的代码, 植入病毒代码, 并重新打成模块发送给你----->你以为是官方的版本

结果黑客就把病毒代码植入到了你的项目中, 你的项目就不安全了, 面临着数据全部泄露的风险。

3).go.sum是如何实现校验机制的?它包含什么内容?

说到校验安全机制, 有一种常规的玩法就是使用不可逆加密算法, 不可逆加密算法是指将a文本通过算法加密成b文本后, b文本永远也不能反着计算出a文本。

不可加密算法的具体是怎么应用的呢? 它是如何起作用的?

我们在这里先讲一个不可逆的加密算法SHA-256算法。

SHA-256算法的功能就是将一个任意长度的字符串转换成一个固定长度为64的字符串, 比如:

4e07408562bedb8b60ce05c1decfe3ad16b72230967de01f640b7e4729b49fce

这里从4e07代表四个字符串, 按此算, 这个加密后的字符串为64个。

为什么是64个呢?

因为64个字符串每两个字符为一组, 比如4e是一组, 07是一组, 也就是说有32组, 每一组是一个十六进制的数值, 一个十六进制的数值也就是两个字符用计算机中的8个字节内存空间存储, 也就是一个十六进制的数字, 有两个字符串, 占8个字节, 一个字节等同8位(bit)(位只能存储0和1两个值), 也就是说:

$32 (32个十六进制数, 每个十六进制数用两个字符表示) * 8字节 = 256位。$

仔细看名字, SHA代表是算法的加密方式类型, 256代表的是他这个是256位的版本。

具体原理实现是SHA内部定义了一系列固定数值的表, 然后加密的时候无论是需要加密多少文字, 它都按照一定的规则从需要加密的文字中按一定规则抽取其中的缩略一部分, 然后拿缩略的一部分和SHA内部的固定数值表进行固定的hash映射和算术操作, 这个hash映射和算术操作的顺序是固定写死的, 公共数据表是写死的, 这个写死的顺序和公共数据表就是这个算法的具体内容本质。

这样的话, 因为抽取的是缩略的内容, 所以我们可以把输出结果固定在64个字符, 256位。

因为是缩略的内容, 所以我们不可能通过缩略的内容反推出完整的结果。

但是, 相同的文本按照这个算法加密出来的64个字符肯定是相同的, 同时, 只要改变原需要加密文本的一个字符, 也会造成加密出来的64个字符大不相同。

我们用SHA-256通常是这么用的:

A方 要 发送信息给 B方

B方 要确定信息是 A 方发送的, 没有经过篡改

此时A和B同时约定一个密码字符串，比如abc。

这个abc只有A方和B方知道。

A 方把 需要传输的文本拼接上abc，然后通过SHA-256加密算出一个值，把原文本和算出的值全部发送给B。

B 方 拿出原文本，拼接上abc，进行SHA256计算，看看结果是否和传输过来的A传输的值一样，如果一样，代表中间没有被篡改。

为什么呢？

因为如果有一个黑客C想要篡改，他就得同时篡改原文本和算出的签名值。

可是C不知道密码是abc，它也就不能把abc拼接到原文后，所以它算出来的签名和B算出来的签名肯定不一致。

所以B如果自己算出的签名值与接收到的签名值不一致，B就知道不是A发过来的，就能校验发送端的源头是否是官方安全的了。

接下来我们讲一下go.sum的验证机制。

首先说下go.sum中存储的内容，这个文件存储的每一行都是如下格式

模块名 版本号 hash签名值

示例：

```
github.com/google/uuid v1.1.1 h1:Gkbcsh/GbpXz7lPftLA3P6TYMwjCLYm83jiFQZF/3gY=
github.com/google/uuid v1.1.1/go.mod
h1:TIyPZe4MgqvfeYDBFedMoGGpEw/LqOeaOT+nhxU+yHo=
```

这里的hash签名值是拿当前模块当前版本号内的所有代码字符串计算出来的一个值，就是通过上面讲解的SHA-256计算的。

所以哪怕是这个模块中的代码有一个字变了，计算出来的hash值也不相同。

第三方模块库在每发布一个新的模块版本后，会按照SHA-256计算出对应版本的hash值，然后提供给外部获取用于检验安全性。

当我们 go mod download 和 go get package@version后 会更新go.mod中的模块路径和版本。

然后会更新或者创建根目录下go.sum文件中的模块名 版本号 和hash值。

在go.sum中的hash值是在下载和更新依赖包的时候，同时获取官方提供的版本号得来的。

也就是说，基本上go.sum中的文件都是从官网（外国）（中国是第三方模块仓库）上获得的正品版本号，这个版本号是仓库方自己算的，你只是获取到了存储到你自己的go.sum中。

具体如何获取版本号有个小知识点：

```
go module
```

机制在下载和更新依赖的时候会取出操作系统中名为`GOSUMDB`的环境变量中的值，这个服务器地址值代表了从哪个第三方仓库获取对应的正品版本号。

重点来了，当你在go build 打包创建go项目的时候，go build的内部指令会去拿你本地的模块文件进行SHA-256计算，然后拿到一个计算出来的结果值，之后它会拿此值和go.sum中的正确的从官网拉取的值进行对比，如果不一样，说明这个模块包不是官方发布的，也就是你本地的模块包和官方发布的模块包中的代码肯定有差异。

四.专门详解Go并发编程相关知识

1.Go为什么天然支持高并发，纤程比线程的优势是什么？

Go语言在设计的时候就考虑了充分利用计算机的多核处理器，具体表现为，Go中开启一个并发的任务以操作系统的线程资源调度为单位的，而是Go的创造者们自己写了一套管理多个任务的机制，在这个机制下，每一个并发的任务线程叫做纤程，这个纤程的作用等同一个线程，也是并发执行的，只不过纤程是在应用程序管理的，懂底层的可以讲是在用户态的一个线程，而Java中调度的线程是属于操作系统，也就是操作系统内核态的线程。

用户态的纤程归属于用户编写的软件管理和调度，优点是可以根据情况灵活实现堆栈的内存分配，最优化其中的运行资源配置。

内核态的线程归属于操作系统调度和管理，他底层是有windows或者linux操作系统底层的代码管理的，那么他就不灵活，每个线程分配的资源可能造成浪费，创建的线程数肯定也有一定的限制。

Go的创造可以为自己的语言和任务灵活配置资源，Linux和windows操作系统的代码是通用的，总不能为你这个语言修改源代码把。

在实际程序运行中，一个操作系统的内核态线程可能管理着好几个甚至数十个纤程(根据实际情况和设置不同而不同)，所以省去了线程时间片上下文切换的时间。

同时因为内部机制灵活，所以执行效率高，占用内存也少。

这就是Go语言的并发优势的核心所在。

2.并发和并行的区别？

并发是指的一个角色在一段时间内通过来回切换处理了多个任务。

并行是指两个或者多个角色同时处理自己的任务。

举例：

并发：在一个小时内，你写了10分钟语文作业，又写了10分钟数学，之后又写了10分中英语作业，然后再从语文10分钟，数学10分钟，英文10分钟又来一次。

这个叫做你并发的写语文数学英语作业。

你一个一段时间（一个小时内）通过切换（一会写数学，一会写语文。。。），处理了多个任务（写了三门课的作业）

并行：你和小明同时写自己的作业。你们俩同时运行的状态叫做并行运作状态，强调的是你们两个人同时在处理任务(做作业)。

你和小明(两个以上的角色)同时写作业(处理自己的任务)。

在计算机中，比如有4个cpu，4个cpu同时工作，叫做这4个cpu并行执行任务，每个cpu通过时间片机制上下文切换处理100个小任务，叫做每个cpu并发的处理100个任务。

3.Go是如何用Channel进行协程间数据通信数据同步的？

go中的线程相关的概念是Goroutines(并发)，是使用go关键字开启。

Java中的线程是通过Thread类开启的。

在go语言中，一个线程就是一个Goroutines，主函数就是（主） main Goroutines。

使用go语句来开启一个新的Goroutines

比如：

普通方法执行

```
myFunction()
```

开启一个Goroutines来执行方法

```
go myFunction()
```

java中是

```
new Thread()->{  
    //新线程逻辑代码  
}.start();
```

参考下面的代码示例：

```
package main  
  
import (  
    "fmt"  
)  
  
//并发开启新线程goroutine测试  
  
//我的方法  
func myFunction() {  
    fmt.Println("Hello!!!")  
}  
//并发执行方法  
func goroutineTestFunc() {  
    fmt.Println("Hello!!! Start Goroutine!!!")  
}  
  
func main() {  
    /*  
    myFunction()  
    //go goroutineTestFunc()  
    //此时因为主线程有时候结束的快，goroutineTestFunc方法得不到输出，由此可以看出是开启了新的  
    线程。  
    */  
    //打开第二段执行  
    /*  
    go goroutineTestFunc()  
    time.Sleep(10*time.Second)//睡一段时间 10秒
```

```
myFunction()  
    */  
}
```

线程间的通信：

java线程间通信有很多方式：

比如最原始的 wait/notify

到使用juc下高并发线程同步容器，同步队列

到CountDownLatch等一系列工具类

.....

甚至是分布式系统不同机器之间的消息中间件，单机的disruptor等等。

Go语言不同，线程间主要的通信方式是Channel。

Channel是实现go语言多个线程（goroutines）之间通信的一个机制。

Channel是一个线程间传输数据的管道，创建Channel必须声明管道内的数据类型是什么

下面我们创建一个传输int类型数据的Channel

代码示例：

```
package main  
  
import "fmt"  
  
func main() {  
    ch := make(chan int)  
    fmt.Println(ch)  
}
```

channel是引用类型，函数传参数时是引用传递而不是值拷贝的传递。

channel的空值和别的应用类型一样是nil。

==可以比较两个Channel之间传输的数据类型是否相等。

channel是一个管道，他可以收数据和发数据。

具体参照下面代码示例：

```
package main  
  
import (  
    "fmt"  
    "time"  
)  
//channel发送数据和接受数据用 <-表示,是发送还是接受取决于chan在 <-左边还是右边  
//创建一个传输字符串数据类型的管道  
var chanStr = make(chan string)  
func main() {  
    fmt.Println("main goroutine print Hello ")
```



```

//默认channel是没有缓存的，阻塞的，也就是说，发送端发送后直到接受端接受到才会施放阻塞往下走。
//同样接收端如果先开启，直到接收到数据才会停止阻塞往下走
//开启新线程发送数据
go startNewGoroutineOne()
//从管道中接收读取数据
go startNewGoroutineTwo()
//主线程等待，要不直接结束了
time.Sleep(100*time.Second)
}

func startNewGoroutineOne() {
    fmt.Println("send channel print Hello ")
    //管道发送数据
    chanStr <- "Hello!!!"
}

func startNewGoroutineTwo(){
    fmt.Println("receive channel print Hello ")
    strVar := <-chanStr
    fmt.Println(strVar)
}

```

无缓存的channel可以起到一个多线程间线程数据同步锁安全的作用。

缓存的channel创建方式是

make(chan string,缓存个数)

缓存个数是指直到多个数据没有消费或者接受后才进行阻塞。

类似于java中的synchronized和lock

可以保证多线程并发下的数据一致性问题。

首先我们看一个线程不安全的代码示例：

```

package main

import (
    "fmt"
    "time"
)

//多线程并发下的不安全问题
//金额
var moneyA int =1000
//添加金额
func subtractMoney(subMoney int) {
    time.Sleep(3*time.Second)
    moneyA-=subMoney
}

//查询金额
func getMoney() int {
    return moneyA;
}

```

```

func main() {

    //添加查询金额
    go func() {
        if(getMoney()>200) {
            subtractMoney(200)
            fmt.Printf("200元扣款成功，剩下：%d元\n",getMoney())
        }
    }()

    //添加查询金额
    go func() {
        if(getMoney()>900) {
            subtractMoney(900)
            fmt.Printf("900元扣款成功，剩下：%d元\n",getMoney())
        }
    }()

    //正常逻辑，只够扣款一单，可以多线程环境下结果钱扣多了
    time.Sleep(5*time.Second)
    fmt.Println(getMoney())
}

```

缓存为1的channel可以作为锁使用：

示例代码如下：

```

package main

import (
    "fmt"
    "time"
)

//多线程并发下使用channel改造
//金额
var moneyA = 1000
//减少金额管道
var synchLock = make(chan int,1)

//添加金额
func subtractMoney(subMoney int) {
    time.Sleep(3*time.Second)
    moneyA-=subMoney
}

//查询金额
func getMoney() int {
    return moneyA;
}

func main() {

```

```

//添加查询金额
go func() {
    synchLock<-10
    if(getMoney()>200) {
        subtractMoney(200)
        fmt.Printf("200元扣款成功, 剩下: %d元\n",getMoney())
    }
    <-synchLock
}()

//添加查询金额
go func() {
    synchLock<-10
    if(getMoney()>900) {
        subtractMoney(900)
        fmt.Printf("900元扣款成功, 剩下: %d元\n",getMoney())
    }
    synchLock<-10
}()
//这样类似于java中的Lock锁, 不会扣多
time.Sleep(5*time.Second)
fmt.Println(getMoney())
}

```

4.Go中的Goroutine使用和GMP模型?

Go中的线程(实际是纤程)goroutine的底层管理和调度是在runtime包中自己实现的, 其中遵循了GMP模型。

G就是一个goroutine, 包括它自身的一些元信息。

M是指操作系统内核态的线程的一个虚拟表示, 一个M就是操作系统内核态的一个线程。

P是一个组列表, P管理着多个goroutines,P还有一些用于组管理的元数据信息。

5.Go的select怎么用?

Go中的select是专门用于支持更好的使用管道(channel)的。

我们之前虽然讲了能从管道中读取数据, 但是这有一个缺陷, 就是我们在一个Goroutine中不能同时处理读取多个channel, 因为在一个Goroutine中, 一个channel阻塞后就无法继续运行了, 所以无法在一个Goroutine处理多个channel,而select很好的解决了这个问题。

select相当于Java中Netty框架的多路复用器的功能。

举例代码示例:

```

package main

import "fmt"

func main() {
    //创建一个缓存为1的chan
    myChan := make(chan int,1)
    for i:=1;i<=100;i++){

```

```

//select 的用法是，从上到下依次判断case 是否可执行，如果可执行，则执行完毕跳出select，
如果不能执行，尝试下一个执行
//这里的可执行是指的不阻塞，也就是说，select从上到下开始挑选一个不阻塞的case执行，执行
完毕后跳出，
//如果所有case都阻塞，则执行default
//如下输出结果，i=奇数的时候走case myChan<-i:，把奇数放入mychan
//走偶数的时候因为myChan中有数据了，则把上一个奇数打印出来。
//所以结果是 1 3 5 7 ...
select {
    case data := <-myChan:
        fmt.Println(data)
    case myChan<-i:
    default:
        fmt.Println("default !!!")
}
}
}

```

6.Go中的互斥锁(类似于Java中的ReentrantLock)

先按线程不安全的数据错误的代码示例：

```

package main

import (
    "fmt"
    "sync"
)

//全局变量
var num int

var wait sync.WaitGroup

func main() {
    wait.Add(5)
    go myAdd()
    go myAdd()
    go myAdd()
    go myAdd()
    go myAdd()
    wait.Wait()
    //预期值等于5万，可是因为线程不安全错误，小于5万
    fmt.Printf("num = %d\n",num)
}

func myAdd() {
    defer wait.Done()
    for i:=0 ;i<10000;i++ {
        num+=1
    }
}

```

打印输出结果：

```
num = 38626
```

互斥锁示例代码如下：

```
package main

import (
    "fmt"
    "sync"
)

//全局变量
var num int

var wait sync.WaitGroup

var lock sync.Mutex
func main() {
    wait.Add(5)
    go myAdd()
    go myAdd()
    go myAdd()
    go myAdd()
    go myAdd()
    wait.Wait()
    //预期值等于5万，可是因为线程不安全错误，小于5万
    fmt.Printf("num = %d\n", num)
}

func myAdd() {
    defer wait.Done()
    for i:=0 ;i<10000;i++ {
        lock.Lock()
        num+=1
        lock.Unlock()
    }
}
```

7.Go中的读写锁(类似于Java中的ReentrantReadWriteLock)

读写锁用于读多写少的情况，多个线程并发读不上锁，写的时候才上锁互斥

读写锁示例代码如下：

```
package main

import (
    "fmt"
    "sync"
    "time"
```

```

)

//金额
var moneyA = 1000
//读写锁
var rwLock sync.RWMutex;
var wait sync.WaitGroup
//添加金额
func subtractMoney(subMoney int) {
    rwLock.Lock()
    time.Sleep(3*time.Second)
    moneyA-=subMoney
    rwLock.Unlock()
}

//查询金额
func getMoney() int {
    rwLock.RLock()
    result := moneyA
    rwLock.RUnlock()
    return result;
}

func main() {
    wait.Add(2)
    //添加查询金额
    go func() {
        defer wait.Done()
        if(getMoney()>200) {
            subtractMoney(200)
            fmt.Printf("200元扣款成功，剩下: %d元\n",getMoney())
        }else {
            fmt.Println("余额不足，无法扣款")
        }
    }()

    //添加查询金额
    go func() {
        defer wait.Done()
        if(getMoney()>900) {
            subtractMoney(900)
            fmt.Printf("900元扣款成功，剩下: %d元\n",getMoney())
        }else {
            fmt.Println("余额不足，无法扣款")
        }
    }()
    wait.Wait()
    fmt.Println(getMoney())
}

```

8.Go中的并发安全Map(类似于CurrentHashMap)

Go中自己通过make创建的map不是线程安全的，具体体现在多线程添加值和修改值下会报如下错误：

```
fatal error : concurrent map writes
```

这个错类似于java中多线程读写线程不安全的容器时报的错。

Go为了解决这个问题，专门给我们提供了一个并发安全的map，这个并发安全的map不用通过make创建，拿来即可用，并且他提供了一些不同于普通map的操作方法。

参考如下代码示例：

```
package main

import (
    "fmt"
    "sync"
)

//创建一个sync包下的线程安全map对象
var myConcurrentMap = sync.Map{}
//遍历数据用的
var myRangeMap = sync.Map{}

func main() {
    //存储数据
    myConcurrentMap.Store(1,"li_ming")
    //取出数据
    name,ok := myConcurrentMap.Load(1)
    if(!ok) {
        fmt.Println("不存在")
        return
    }
    //打印值 li_ming
    fmt.Println(name)
    //该key有值,则ok为true,返回它原来存在的值，不做任何操作；该key无值，则执行添加操作，ok为false,返回新添加的值
    name2, ok2 := myConcurrentMap.LoadOrStore(1,"xiao_hong")
    //因为key=1存在，所以打印是 li_ming true
    fmt.Println(name2,ok2)
    name3, ok3 := myConcurrentMap.LoadOrStore(2,"xiao_hong")
    //因为key=2不存在，所以打印是 xiao_hong false
    fmt.Println(name3,ok3)
    //标记删除值
    myConcurrentMap.Delete(1)
    //取出数据
    //name4,ok4 := myConcurrentMap.Load(1)
    //if(!ok4) {
    //    fmt.Println("name4=不存在")
    //    return
    //}
    //fmt.Println(name4)

    //遍历数据
```

```

    rangeFunc()
}
//遍历
func rangeFunc(){
    myRangeMap.Store(1,"xiao_ming")
    myRangeMap.Store(2,"xiao_li")
    myRangeMap.Store(3,"xiao_ke")
    myRangeMap.Store(4,"xiao_lei")

    myRangeMap.Range(func(k, v interface{}) bool {
        fmt.Println("data_key_value = :",k,v)
        //return true代表继续遍历下一个, return false代表结束遍历操作
        return true
    })
}
}

```

9.Go中的AtomicXXX原子操作类(类似于Java中的AtomicInteger之类的)

Go中的atomic包里面的功能和Java中的Atomic一样，原子操作类，原理也是cas,甚至提供了cas的api函数，这里不做过多讲解，

简单举一个代码示例，因为方法太多，详细的请参考api文档中的atomic包：

```

package main

import "sync/atomic"

func main() {
    //简单举例
    var num int64 = 20
    atomic.AddInt64(&num,1)
}

```

10.Go中的WaitGroup(类似于Java中的CountDownLatch)

现在让我们看一个需求，比如我们开启三个并发任务，然后三个并发任务执行处理完毕后我们才让主线程继续往下面走。

这时候肯定不能用睡眠了，因为不知道睡眠多长时间。

这是Go中的sync包提供了一个WaitGroup的工具，他基本上和Java中的CountDownLatch的功能一致。

接下来让我们看代码示例：

```

package main

import (
    "fmt"
    "sync"
    "time"
)

//获取类似于CountDownLatch的对象

```



```
var wait sync.WaitGroup

func main() {
    //设置计数器任务为3，当3个任务全部done后，wait.Wait()才会松开阻塞
    wait.Add(3)
    go myFun1()
    go myFun2()
    go myFun3()
    //阻塞
    wait.Wait()
}

func myFun1() {
    //计数器减1
    defer wait.Done()
    //睡眠五秒
    time.Sleep(time.Second*5)
    fmt.Println("fun1执行完毕")
}

func myFun2() {
    //计数器减1
    defer wait.Done()
    //睡眠五秒
    time.Sleep(time.Second*5)
    fmt.Println("fun2执行完毕")
}

func myFun3() {
    //计数器减1
    defer wait.Done()
    //睡眠五秒
    time.Sleep(time.Second*5)
    fmt.Println("fun3执行完毕")
}
```